

EECS 836: Machine Learning

Zijun Yao

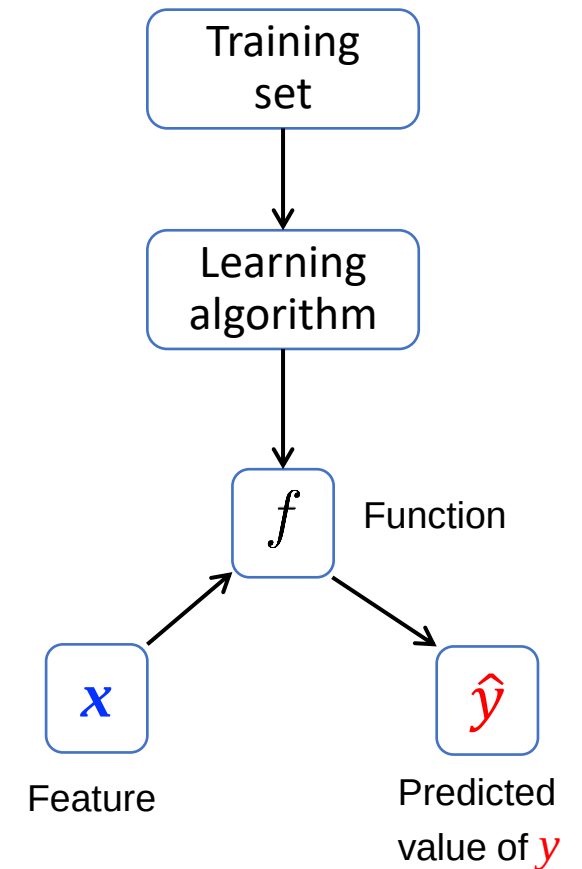
Assistant Professor, EECS Department
The University of Kansas

Agenda

- Linear Regression model
 - Model definition
 - Loss function
 - Optimizing parameters

Supervised learning setup

- Given a collection of records (training set)
 - Each record is characterized by a pair $(\mathbf{x}, y)$
 - $\mathbf{x}$: feature, attribute, independent variable
 - y : target, label, dependent variable
- Goal
 - Learn a model (function $f : \mathcal{X} \rightarrow \mathcal{Y}$) so that $f(\mathbf{x})$ can correctly predict $\hat{y}$ for the corresponding value of y
- Tasks
 - Regression: to predict a continuous value $\hat{y}$
 - Classification: to predict a category class $\hat{y}$



House price prediction - regression

Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Price of House
950	2.5	2	1	\$127,325
1,535	1.5	2	2	\$156,570
1,605	2.25	3	1.5	\$158,895
1,905	2.5	2	1.5	\$200,025
2,057	2.25	3	2	\$230,384
2,227	2.75	3	2	\$233,835
3,150	1	4	2	\$261,420
3,620	3	4	3	\$433,500

Zillow

Save Share More

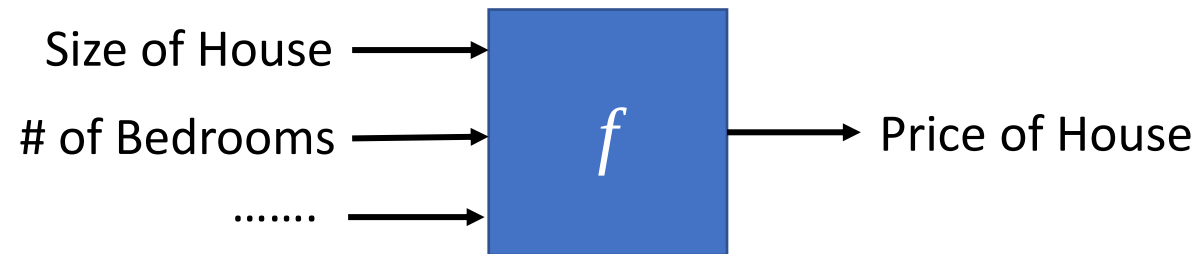
\$439,900 3 bd | 3 ba | 2,031 sqft

3706 Tucker Trl, Lawrence, KS 66049



Facts and features

- Single family residence
- 3 bedrooms
- 3 bathrooms
- Built in 2021
- Forced air gas
- Central air
- 2 Attached garage spaces
- \$1,600 annually HOA fee
- No data
- \$217 price/sqft

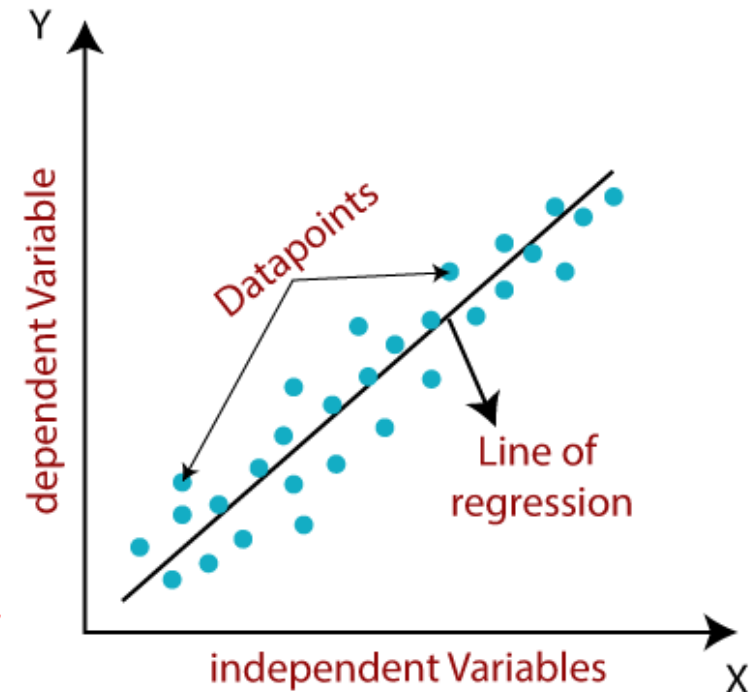


Linear regression

Size of House	Lot Size (acre)	# of Bedrooms	# of Bathrooms	Price of House
950	2.5	2	1	\$127,325
1,535	1.5	2	2	\$156,570
1,605	2.25	3	1.5	\$158,895
1,905	2.5	2	1.5	\$200,025
2,057	2.25	3	2	\$230,384
2,227	2.75	3	2	\$233,835
3,150	1	4	2	\$261,420
3,620	3	4	3	\$433,500

Independent variables (features) x

Dependent variables (targets) y

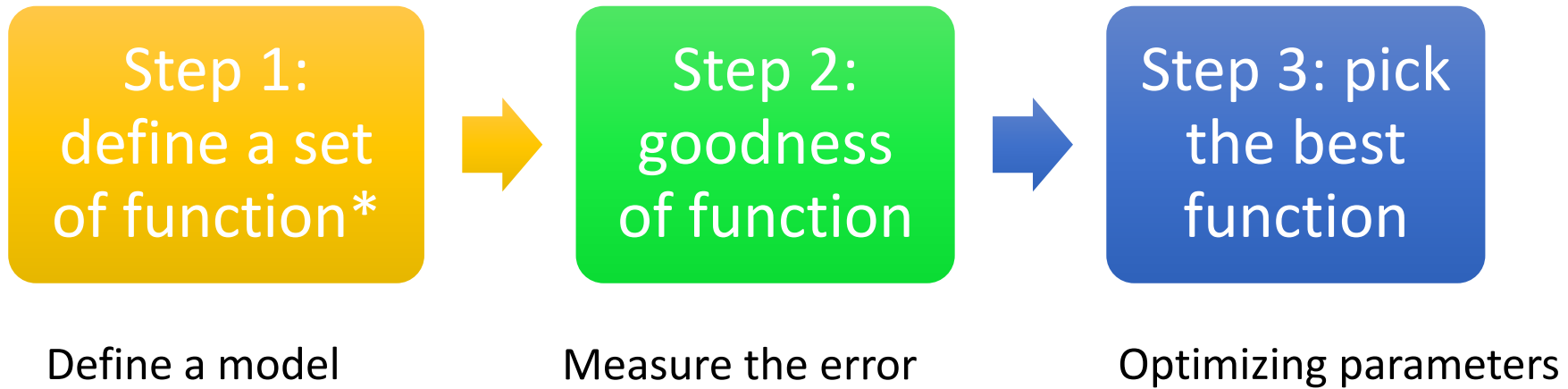


- Given

- Data $\mathbf{X} = \{x^{(1)}, \dots, x^{(n)}\}$ where $x^{(i)} \in \mathbb{R}^d$
- Corresponding labels $\mathbf{y} = \{y^{(1)}, \dots, y^{(n)}\}$ where $y^{(i)} \in \mathbb{R}$

- Goal: find a continuous function that models the continuous points

3 ML steps for linear regression

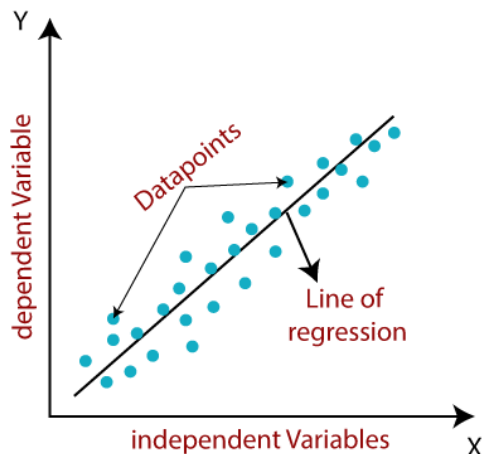


*A set of function means same model but with different values of parameter.

Step 1: Model definition

$$\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Predicted value of y



Price of House
\$127,325
\$156,570
\$158,895
\$200,025
\$230,384
\$233,835
\$261,420
\$433,500

Target

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Data

A linear relationship between feature and target

Step 1: Model definition

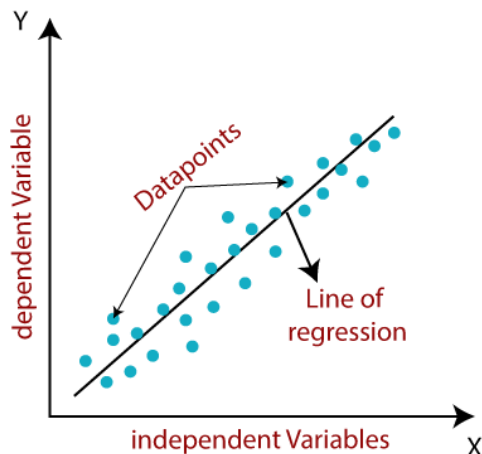
Parameters

Bias: a fixed offset

Weights: significance of each feature

$$\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Predicted value of y



Price of House
\$127,325
\$156,570
\$158,895
\$200,025
\$230,384
\$233,835
\$261,420
\$433,500

Target

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Data

A linear relationship between feature and target

Step 1: Define set of functions

Size of
house



of bath



$f($



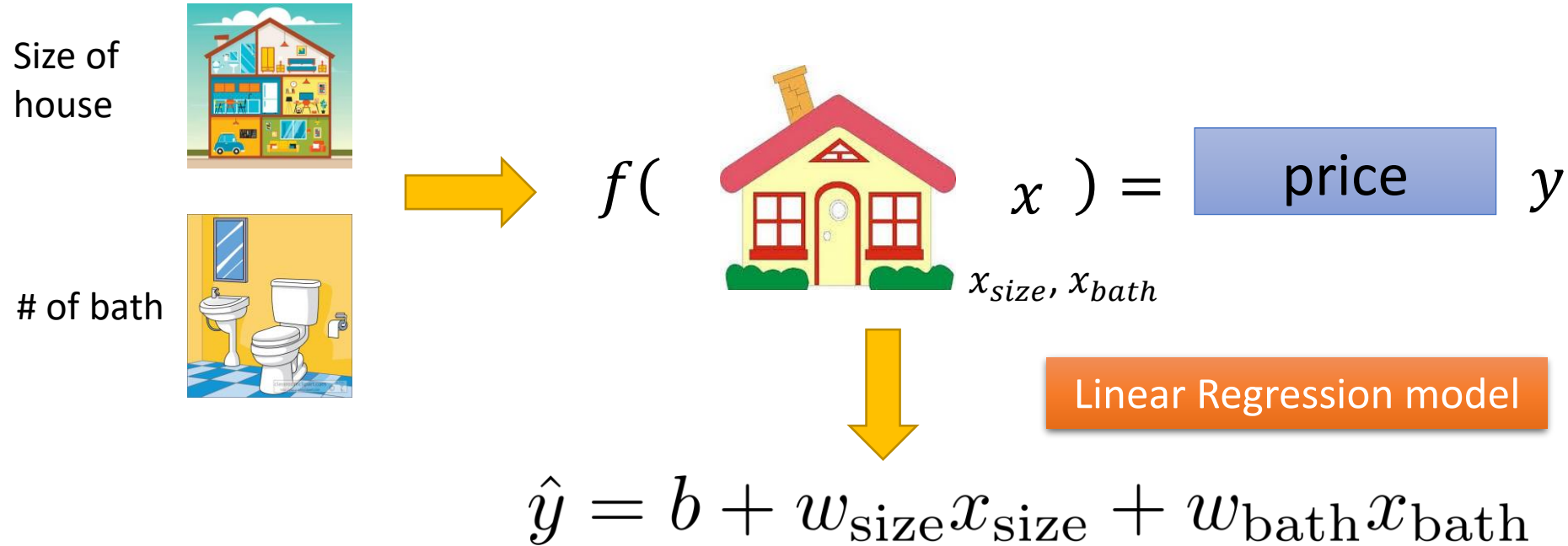
x_{size}, x_{bath}

$) =$

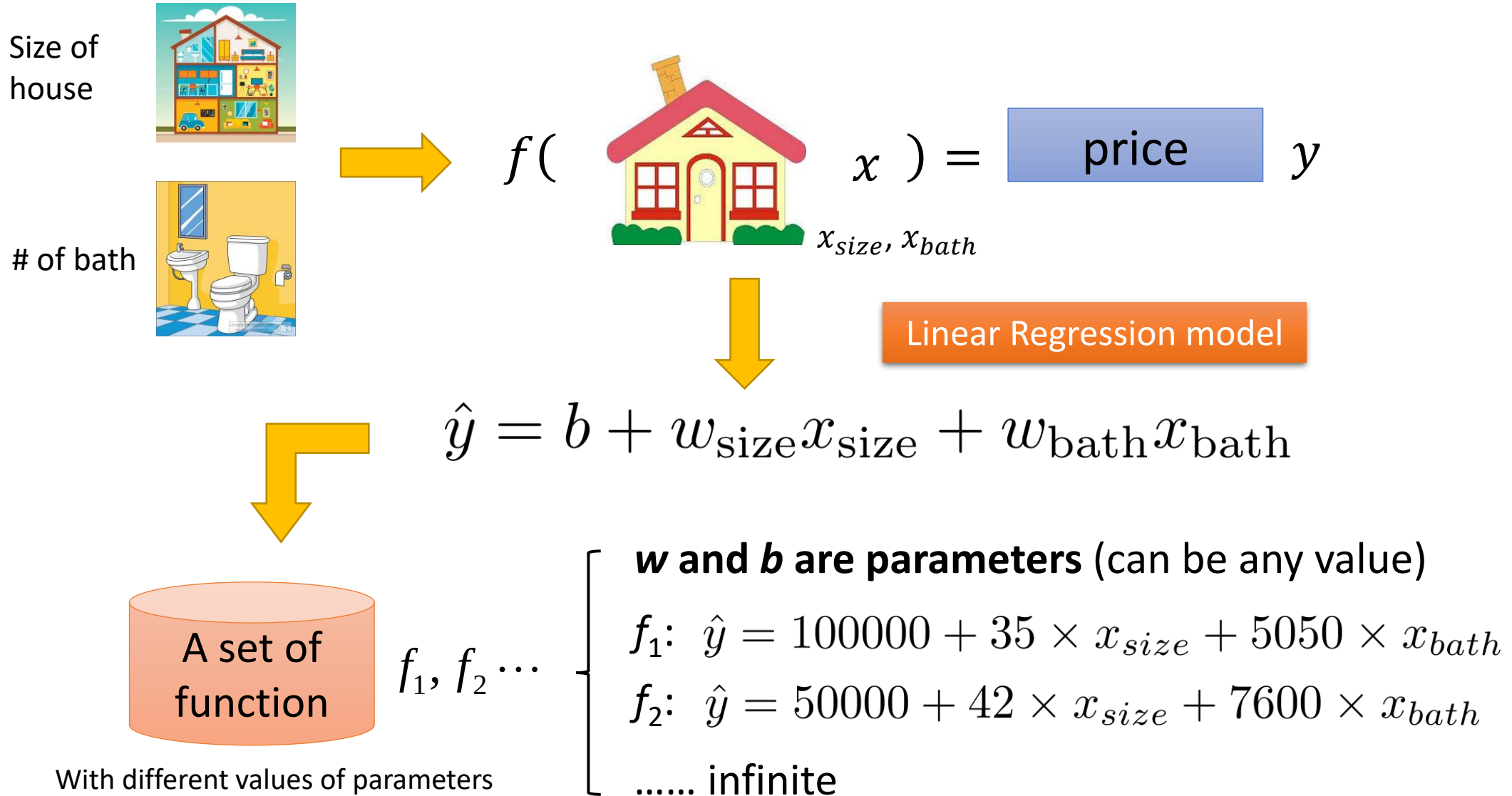
price

y

Step 1: Define set of functions

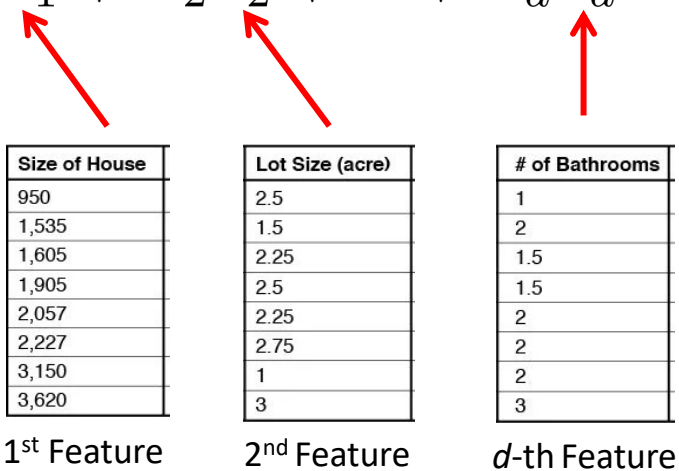


Step 1: Define set of functions



Step 1: A variant form of linear model

$$\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$$



Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Step 1: A variant form of linear model

$$\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature



Equivalence
by $b = w_0$

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Dummy
1
1
1
1
1
1
1
1

0th Feature

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Step 1: A variant form of linear model

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$



Dummy
1
1
1
1
1
1
1
1
1

0st Feature

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Linear Regression model

Prediction

Parameters

Function of $\mathcal{X}$

Features

$$f(x) = \hat{y} = \sum_{i=0}^d w_i x_i$$

Agenda

- Linear Regression model
 - Model definition
 - Loss function
 - Optimizing parameters

Step 2: Goodness of function

How good is a function? - measure the difference between predicted and true y

Model

A set of
function

$f_1, f_2 \dots$

Training
Data

Step 2: Goodness of function

How good is a function? - measure the difference between predicted and true y

Model

A set of
function

$f_1, f_2 \dots$

Training
Data



function
input:



$x_{size}^{(1)} : 4,043$ sqft
 $x_{age}^{(1)} : 26$ years

function
Output (scalar): label value:

$$\hat{y}^{(1)} = f(\mathbf{x}^{(1)})$$

$y^{(1)}$ **784,000**



$x_{size}^{(2)} : 4,976$ sqft
 $x_{age}^{(2)} : 8$ years

$$\hat{y}^{(2)} = f(\mathbf{x}^{(2)})$$

$y^{(2)}$ **724,900**

Suppose we have two house features in
this data: size and age

Superscript means the data index

Step 2: Goodness of function

How good is a function? - measure the difference between predicted and true y

Model

A set of
function

$f_1, f_2 \dots$

Training
Data



function
input:



$x_{size}^{(1)} : 4,043$ sqft
 $x_{age}^{(1)} : 26$ years



$x_{size}^{(2)} : 4,976$ sqft
 $x_{age}^{(2)} : 8$ years

function
Output (scalar): label value:

Measure the difference

$$\hat{y}^{(1)} = f(\mathbf{x}^{(1)})$$

$y^{(1)}$ **784,000**

$$\hat{y}^{(2)} = f(\mathbf{x}^{(2)})$$

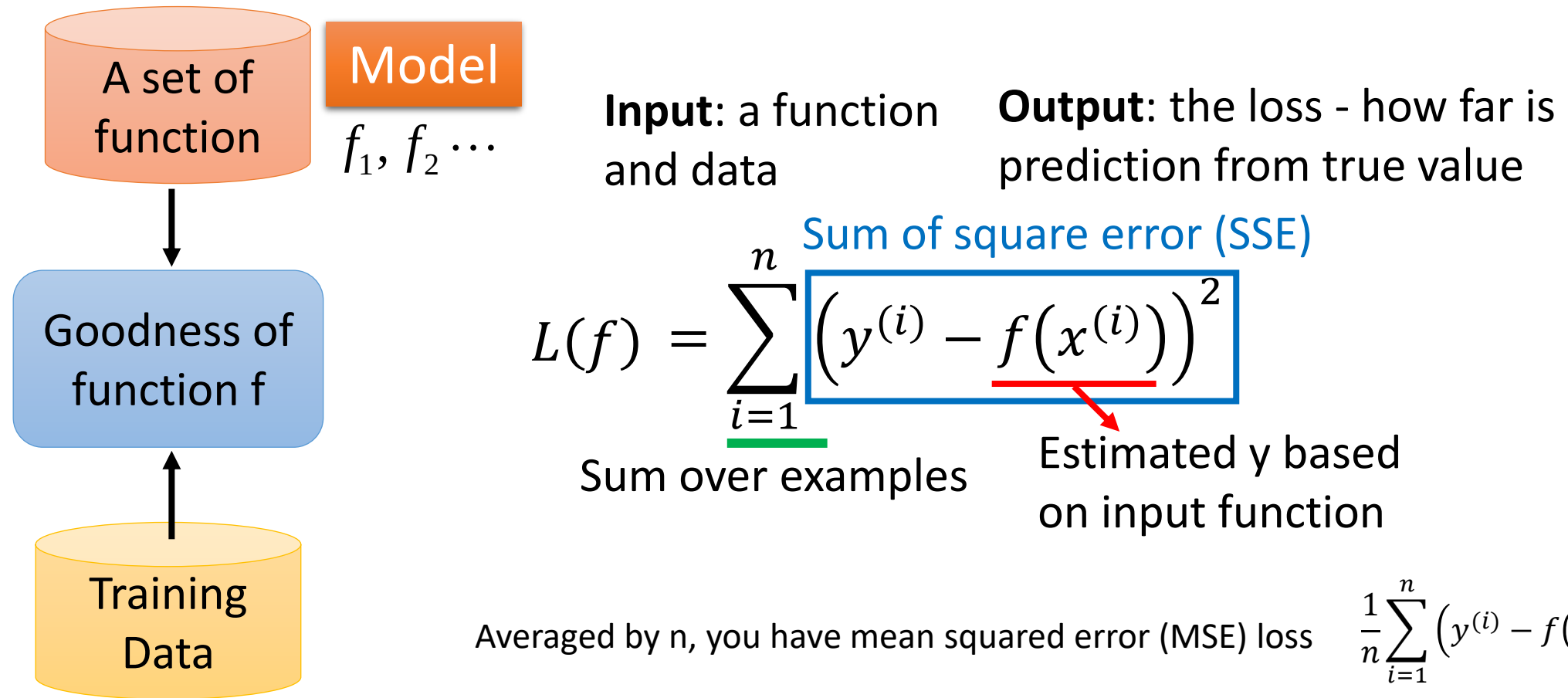
$y^{(2)}$ **724,900**

Suppose we have two house features in
this data: size and age

Superscript means the data index

Step 2: Measure error

How good is a function? - **Loss function** L



Loss function also called cost function and objective function

Step 2: Measure error

How good is a function? - **Loss function** L

Input: a function and data

output: the loss - how far is prediction from true value

$$L(f) = \sum_{i=1}^n \left(\underline{y^{(i)}} - \underline{f(x^{(i)})} \right)^2$$

Sum over examples

True y

Estimated y based
on input function

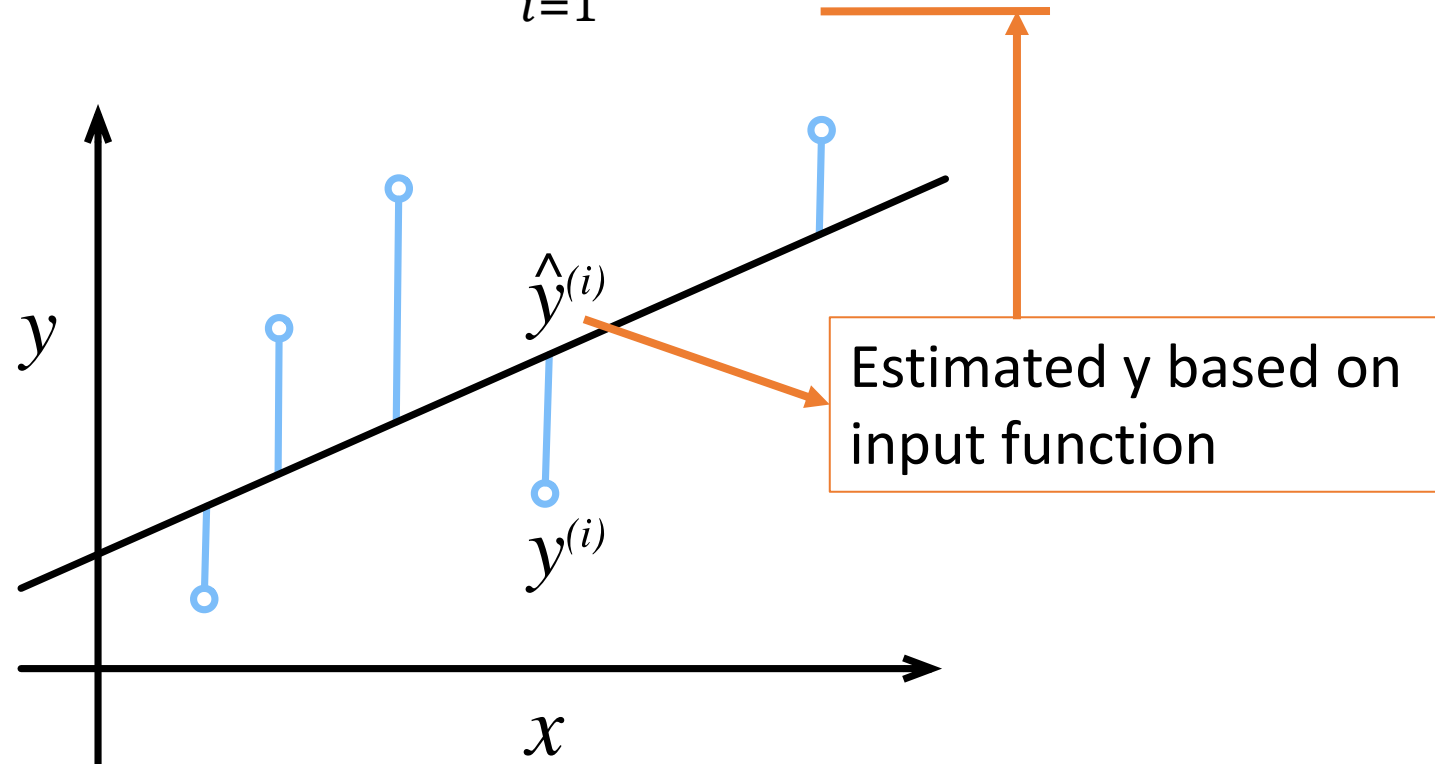


$$L(w_{size}, w_{age}, b) = \sum_{i=1}^n \left(\underline{y^{(i)}} - \underline{\left(b + w_{size} \cdot x_{size}^{(i)} + w_{age} \cdot x_{age}^{(i)} \right)} \right)^2$$

Loss function also called cost function and objective function

Step 2: Loss function

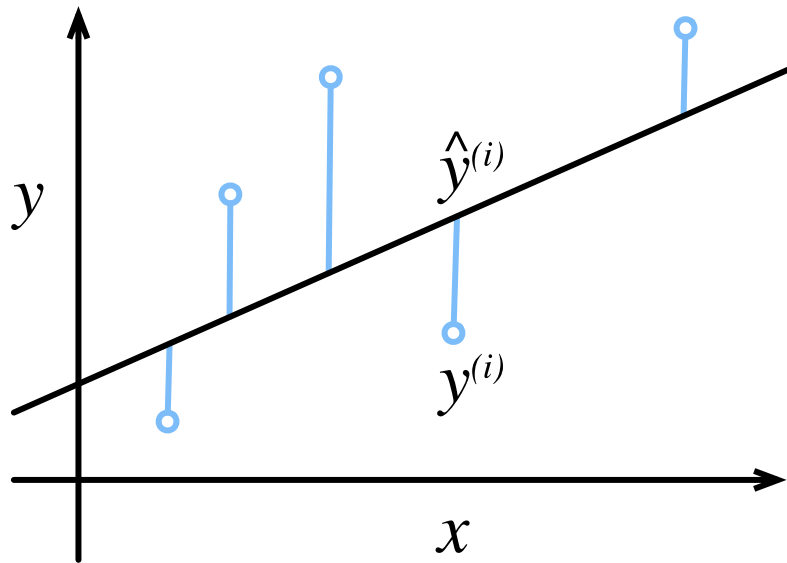
$$L(f) = \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2$$



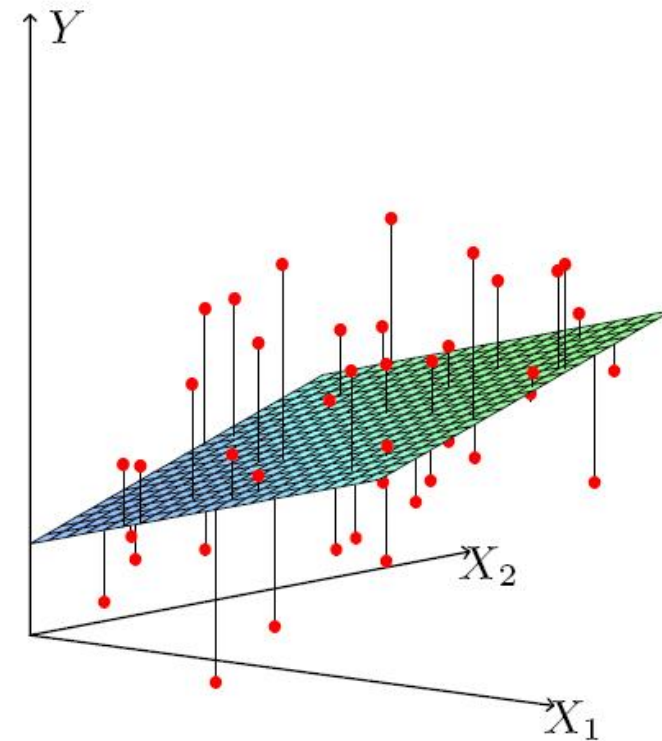
A simple case where only one feature is used to predict y

Step 2: Loss function

$$L(f) = \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2$$



A simple case where only one feature is used to predict y

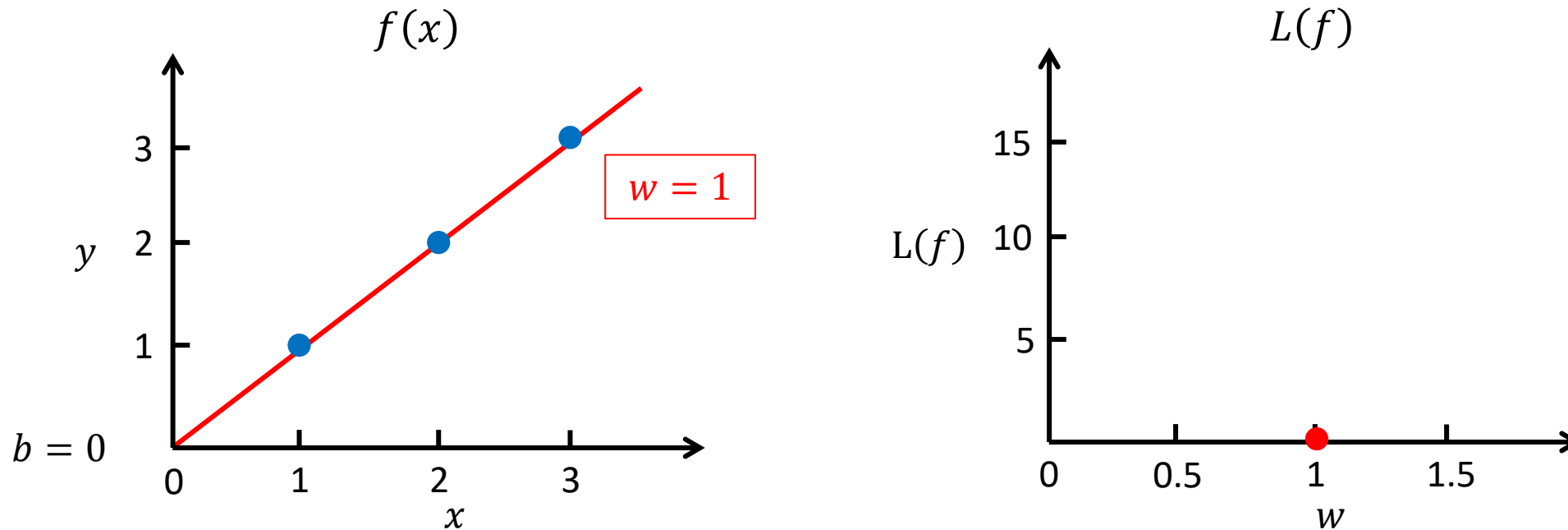


When there are 2 features to predict y

Step 2: Intuition of loss function

- Let's use a simple case with only one feature

$$L(f) = \sum_{i=1}^n (y^{(i)} - w \cdot x^{(i)})^2$$

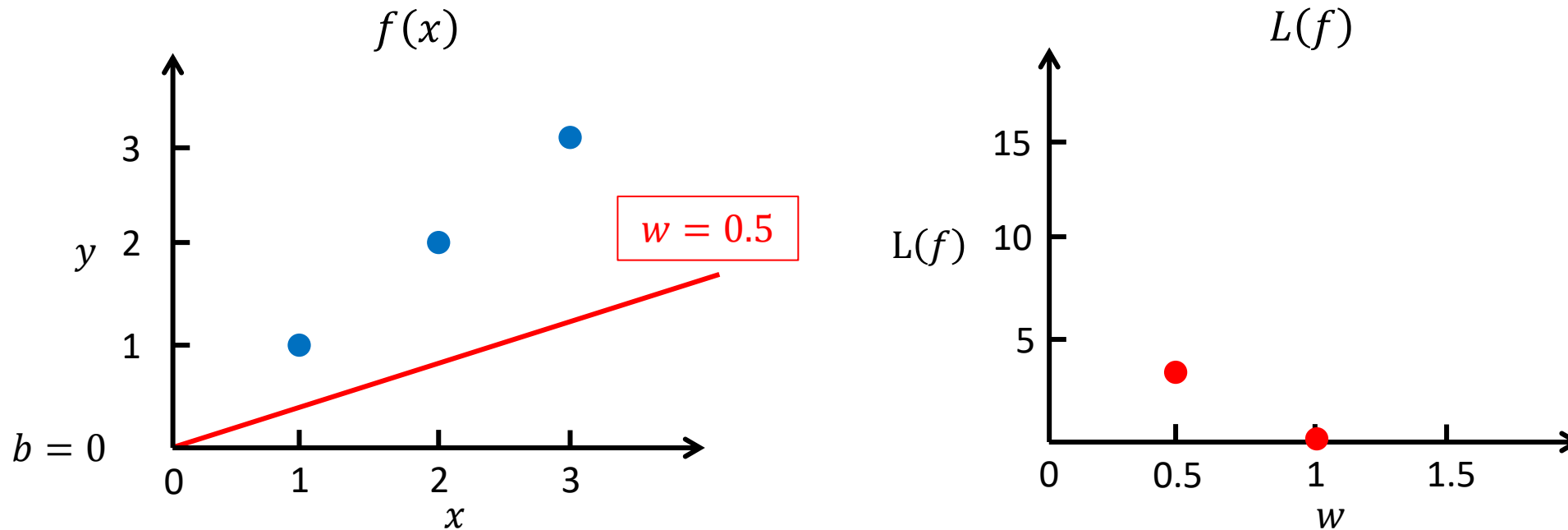


$$L(f) = (1-1)^2 + (2-2)^2 + (3-3)^2 = 0$$

Step 2: Intuition of loss function

- Let's use a simple case with only one feature

$$L(f) = \sum_{i=1}^n (y^{(i)} - w \cdot x^{(i)})^2$$

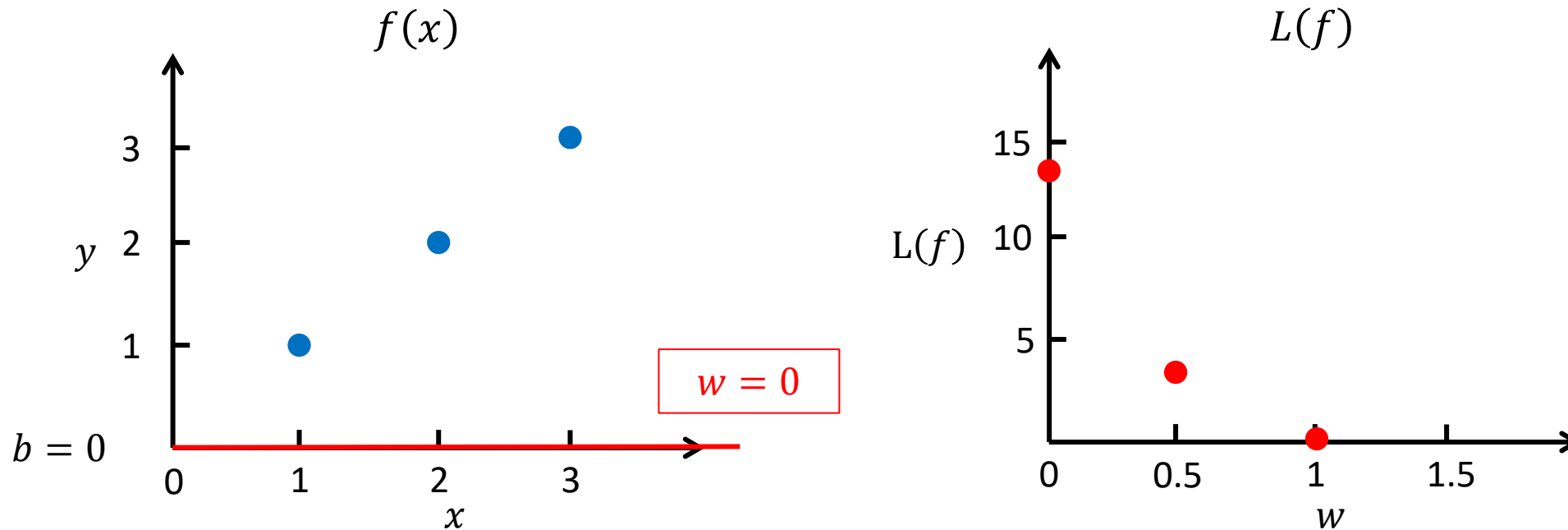


$$L(f) = (1-0.5)^2 + (2-1)^2 + (3-1.5)^2 = 3.5$$

Step 2: Intuition of loss function

- Let's use a simple case with only one feature

$$L(f) = \sum_{i=1}^n (y^{(i)} - w \cdot x^{(i)})^2$$

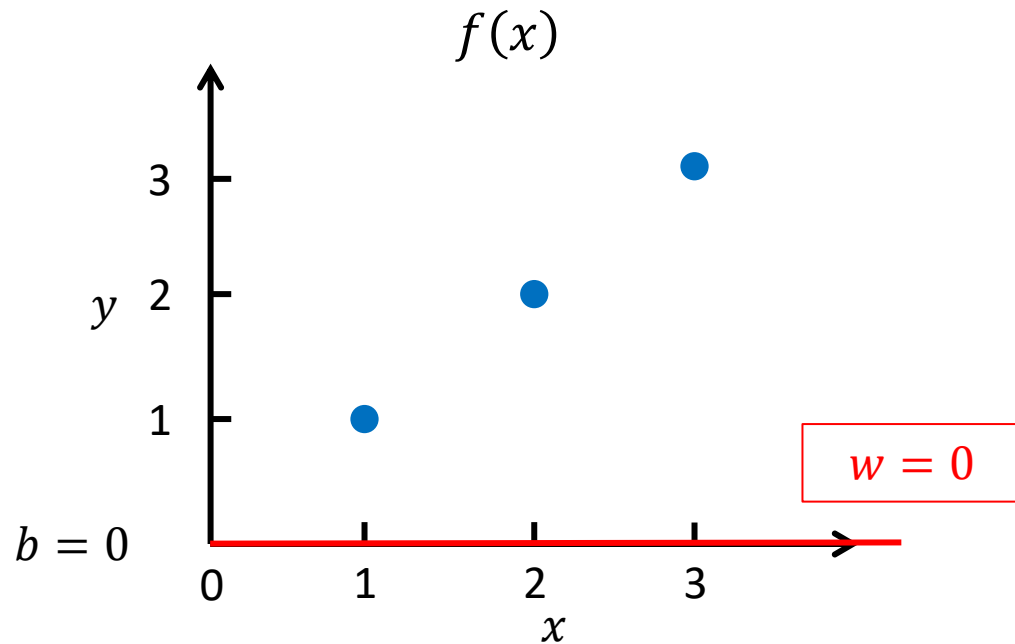


$$L(f) = (1-0)^2 + (2-0)^2 + (3-0)^2 = 14$$

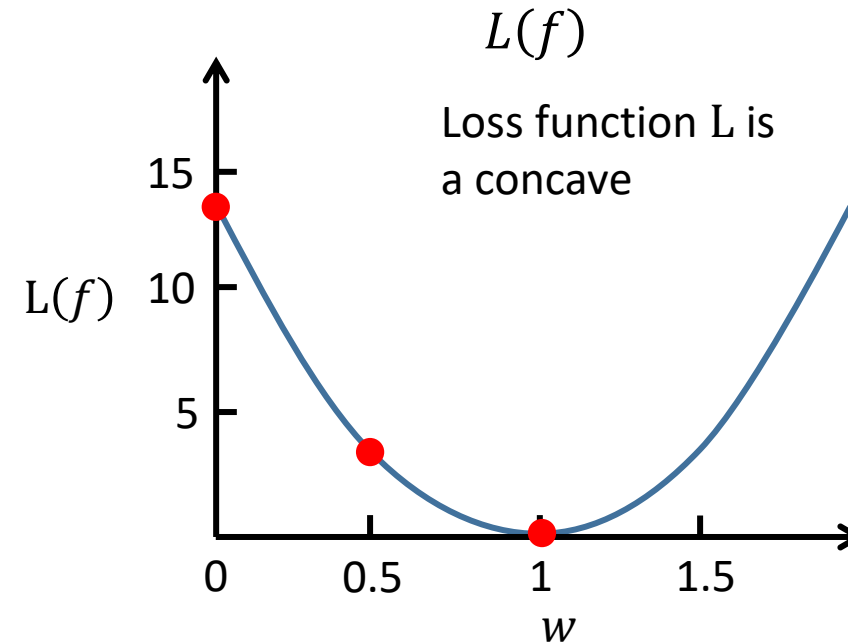
Step 2: Intuition of loss function

- Let's use a simple case with only one feature

$$L(f) = \sum_{i=1}^n (y^{(i)} - w \cdot x^{(i)})^2$$

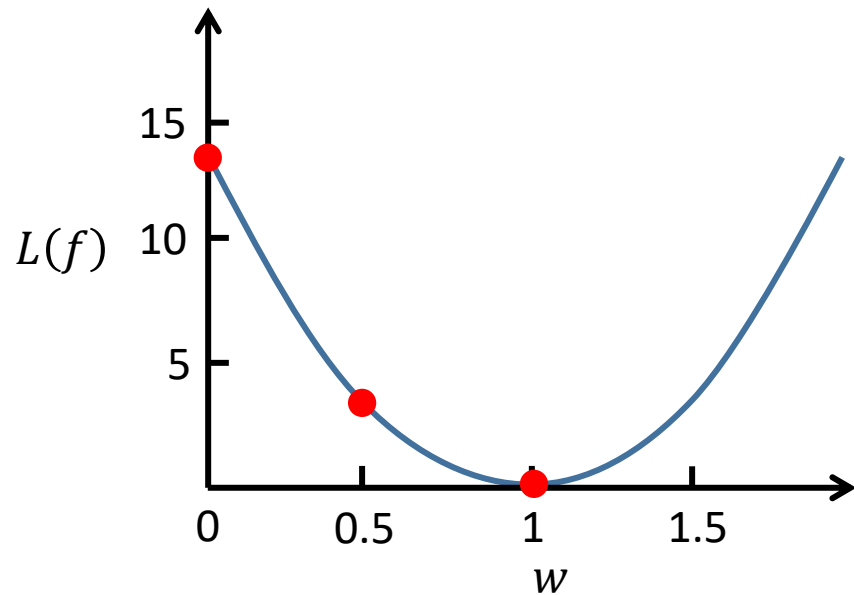


$$L(f) = (1-0)^2 + (2-0)^2 + (3-0)^2 = 14$$

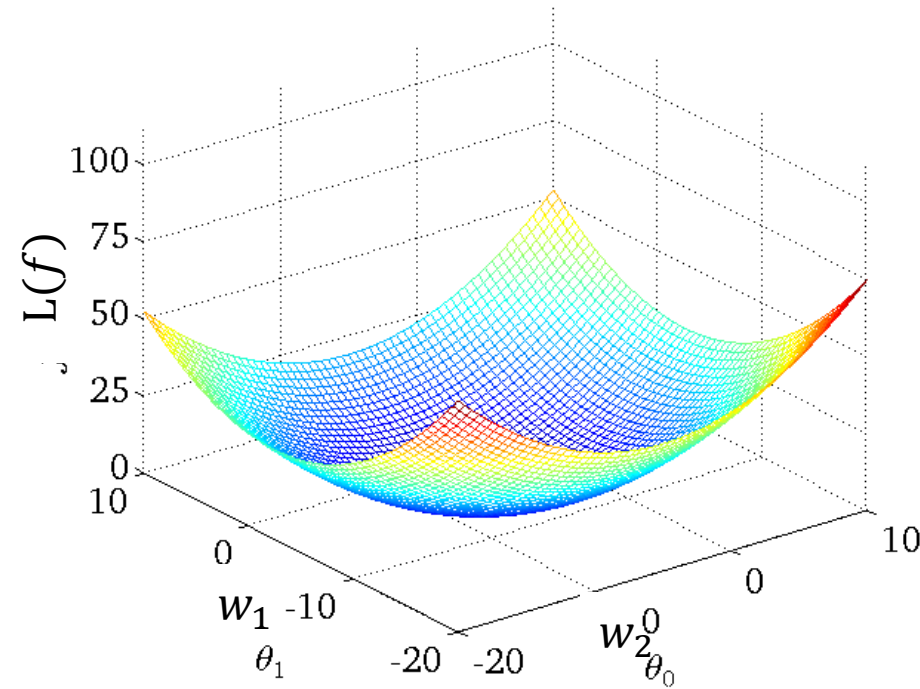


Step 2: Intuition of loss function

One-feature case



Two-feature case

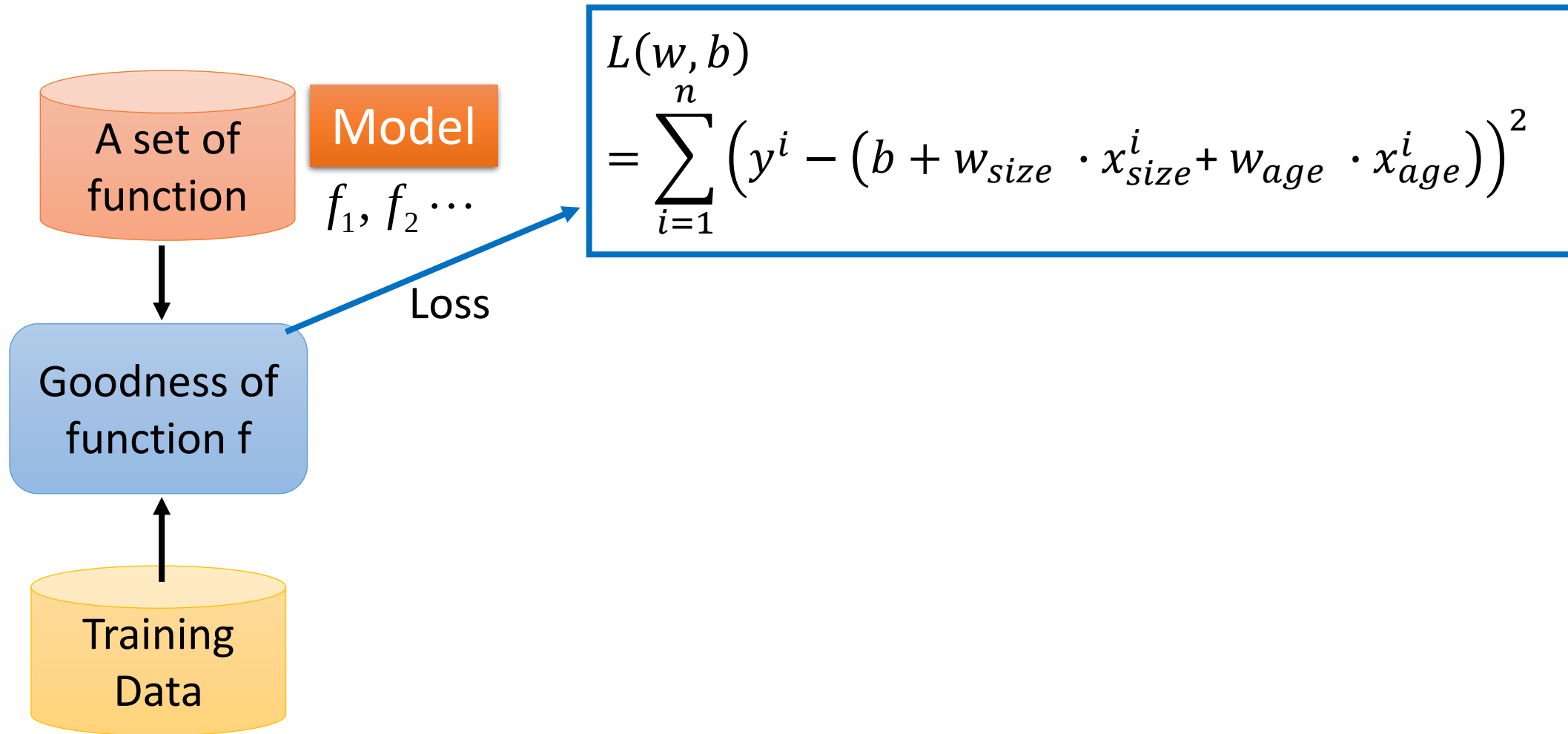


Loss function tracks the performance of model as parameters change

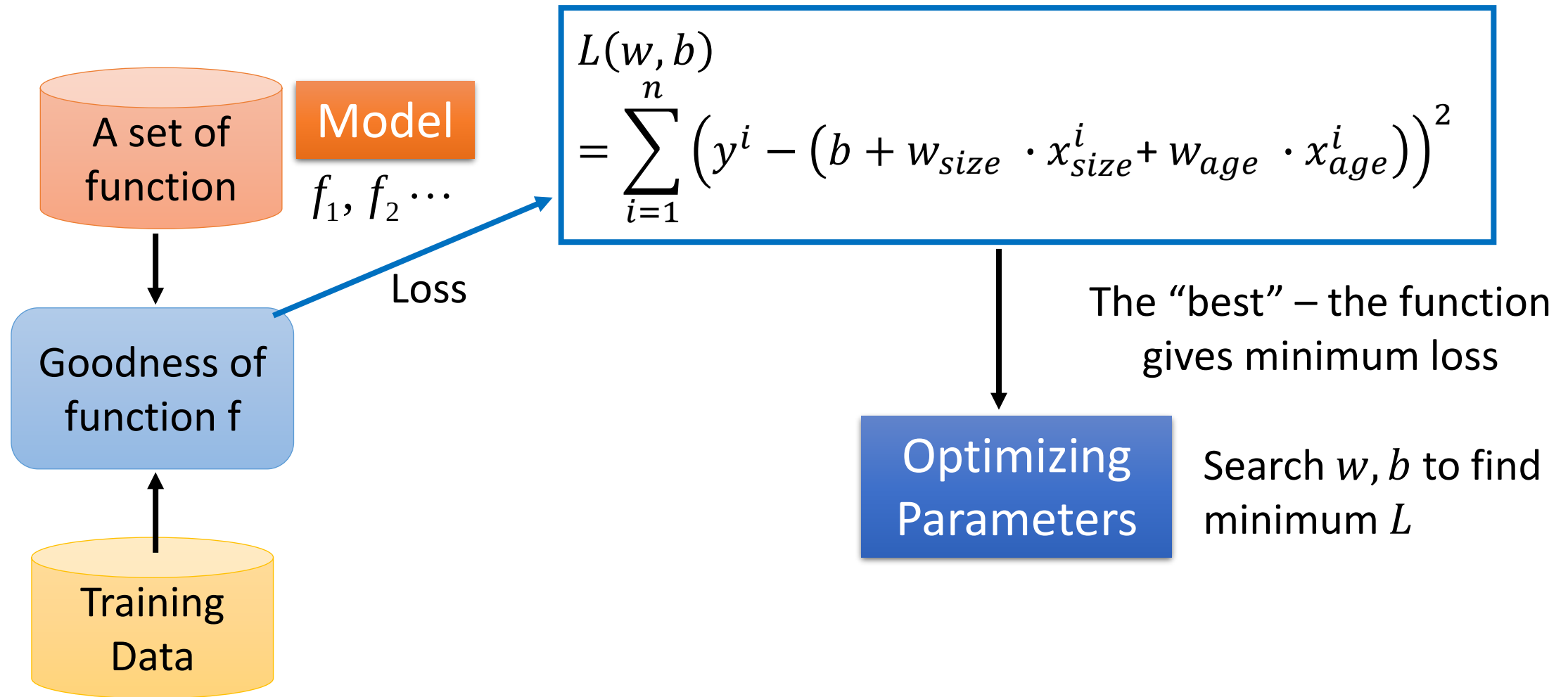
Agenda

- Linear Regression model
 - Model definition
 - Loss function
 - Optimizing parameters

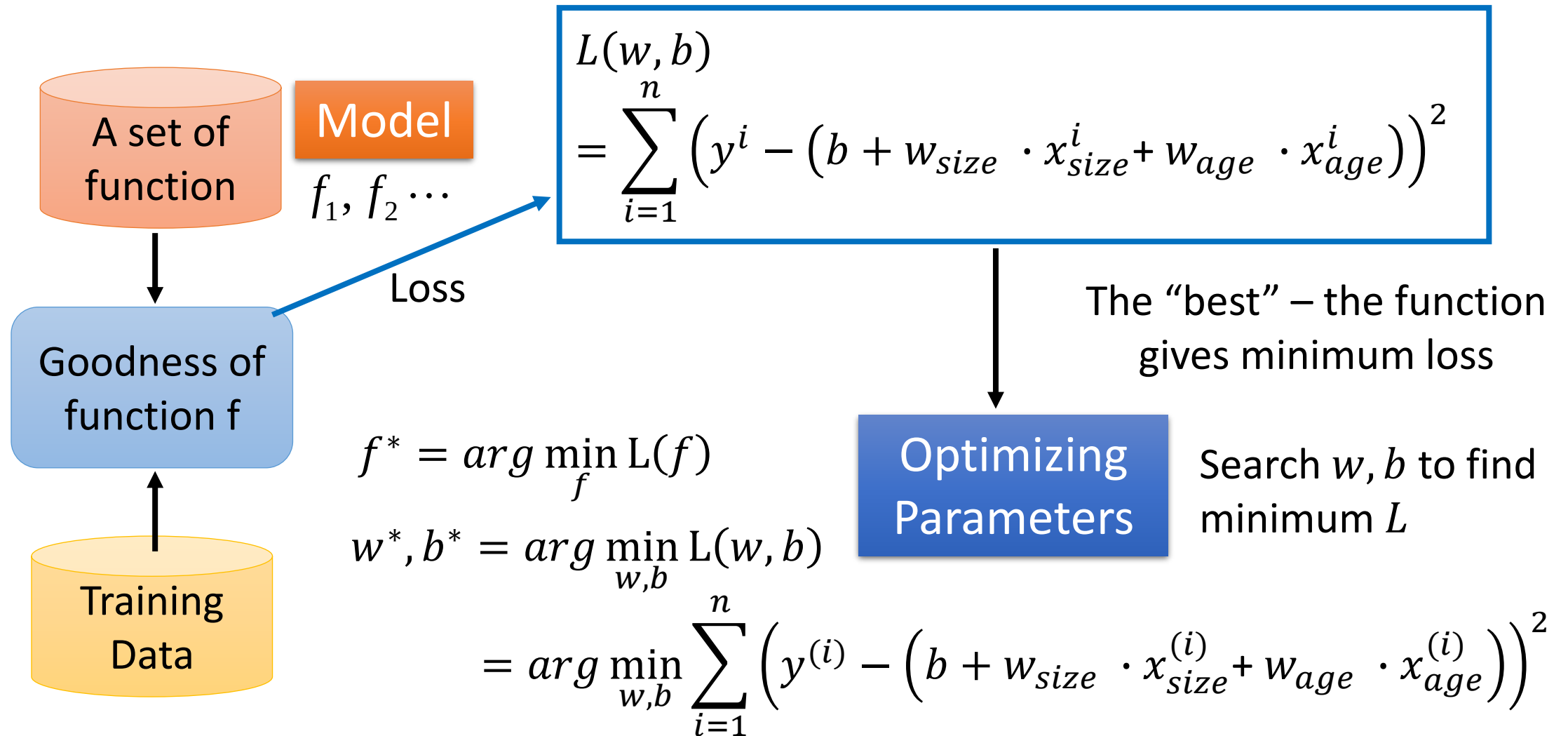
Step 3: Find the best function



Step 3: Find the best function



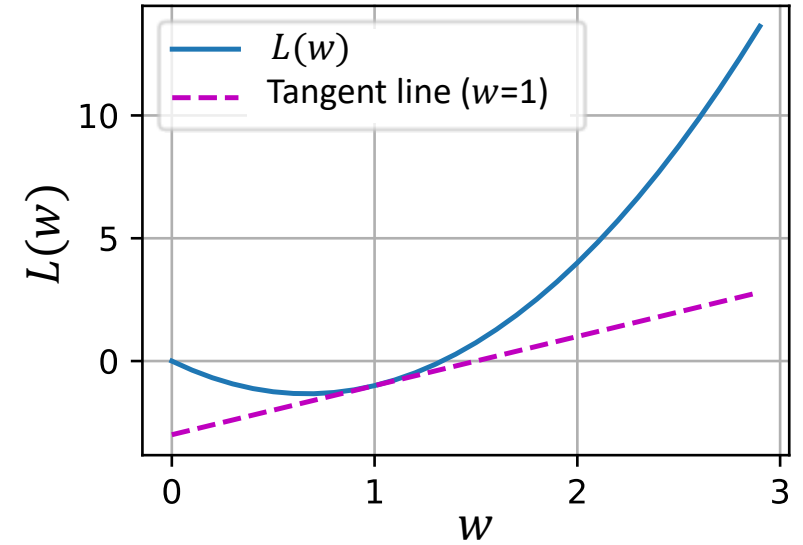
Step 3: Find the best function



Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



The sensitivity to change of loss function with respect to a change in a parameter w

Derivatives

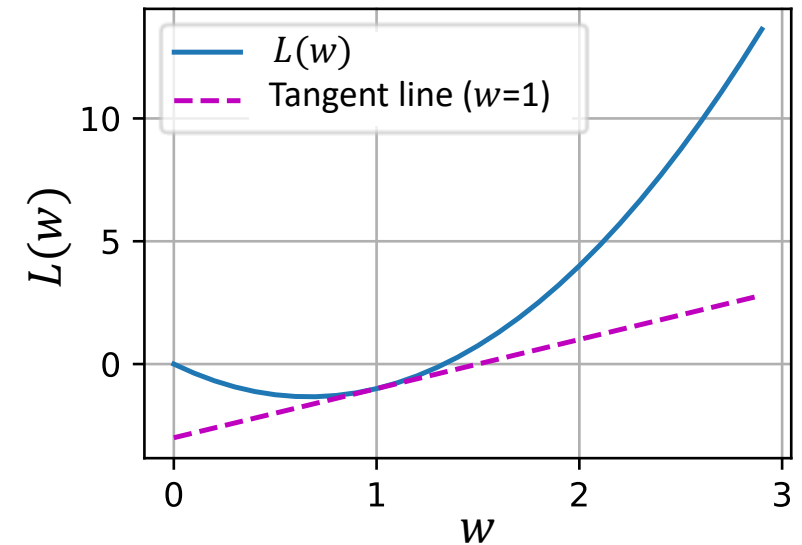
- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

- Partial Derivatives: Let $L(\mathbf{w})$ be loss function of parameters

$$\frac{\partial L(\mathbf{w})}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, w_i + h, w_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, w_i, w_{i+1}, \dots, w_d)}{h}$$

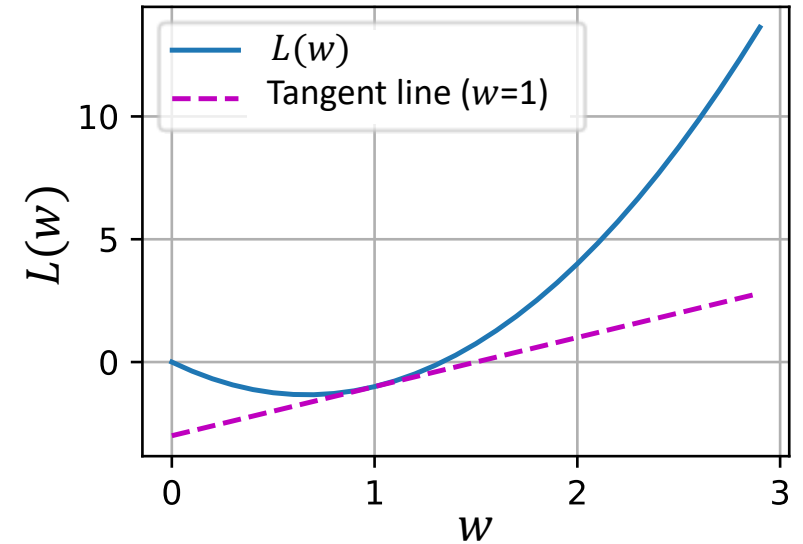
Function derivative with respect to one of parameters w_i , with the others held constant



Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



- Partial Derivatives: Let $L(\mathbf{w})$ be loss function of parameters

$$\frac{\partial L(\mathbf{w})}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, w_i + h, w_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, w_i, w_{i+1}, \dots, w_d)}{h}$$

- Gradients

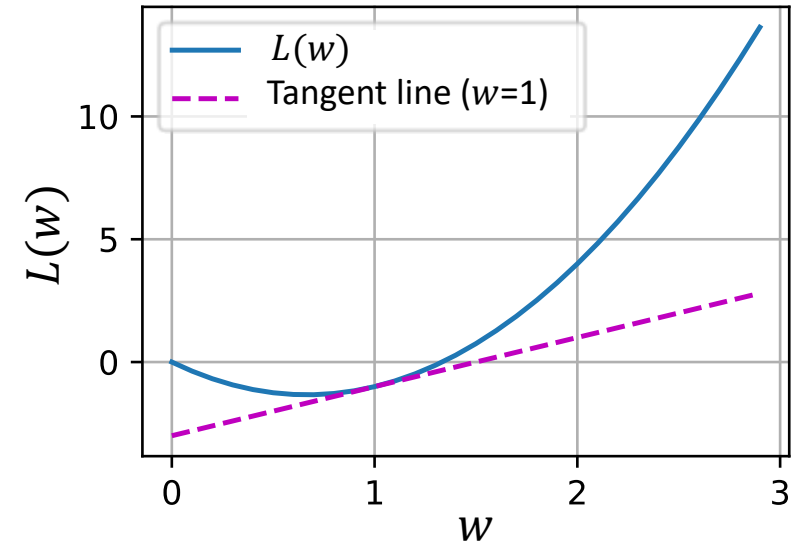
$$\nabla L(\mathbf{w}) = \left[\frac{\partial L(\mathbf{w})}{\partial w_0}, \frac{\partial L(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial L(\mathbf{w})}{\partial w_d} \right]^T$$

Gradients is a vector consists of partial derivative of each parameter

Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



- Partial Derivatives: Let $L(\mathbf{w})$ be loss function of parameters

$$\frac{\partial L(\mathbf{w})}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, w_i + h, w_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, w_i, w_{i+1}, \dots, w_d)}{h}$$

- Gradients

$$\nabla L(\mathbf{w}) = \left[\frac{\partial L(\mathbf{w})}{\partial w_0}, \frac{\partial L(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial L(\mathbf{w})}{\partial w_d} \right]^T$$

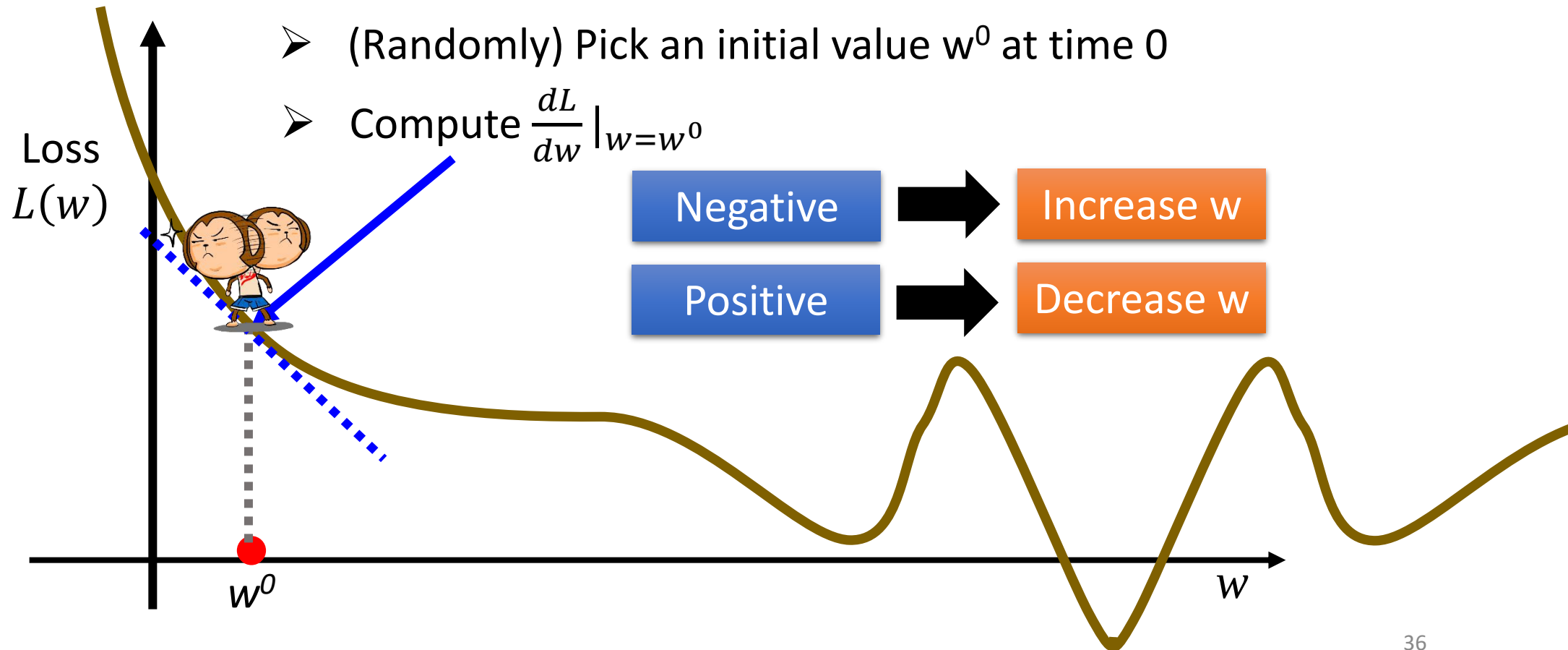
Gradient is a direction that increase the value of $L(\mathbf{w})$

How to reduce Loss function?

Subtract gradient from each parameter w

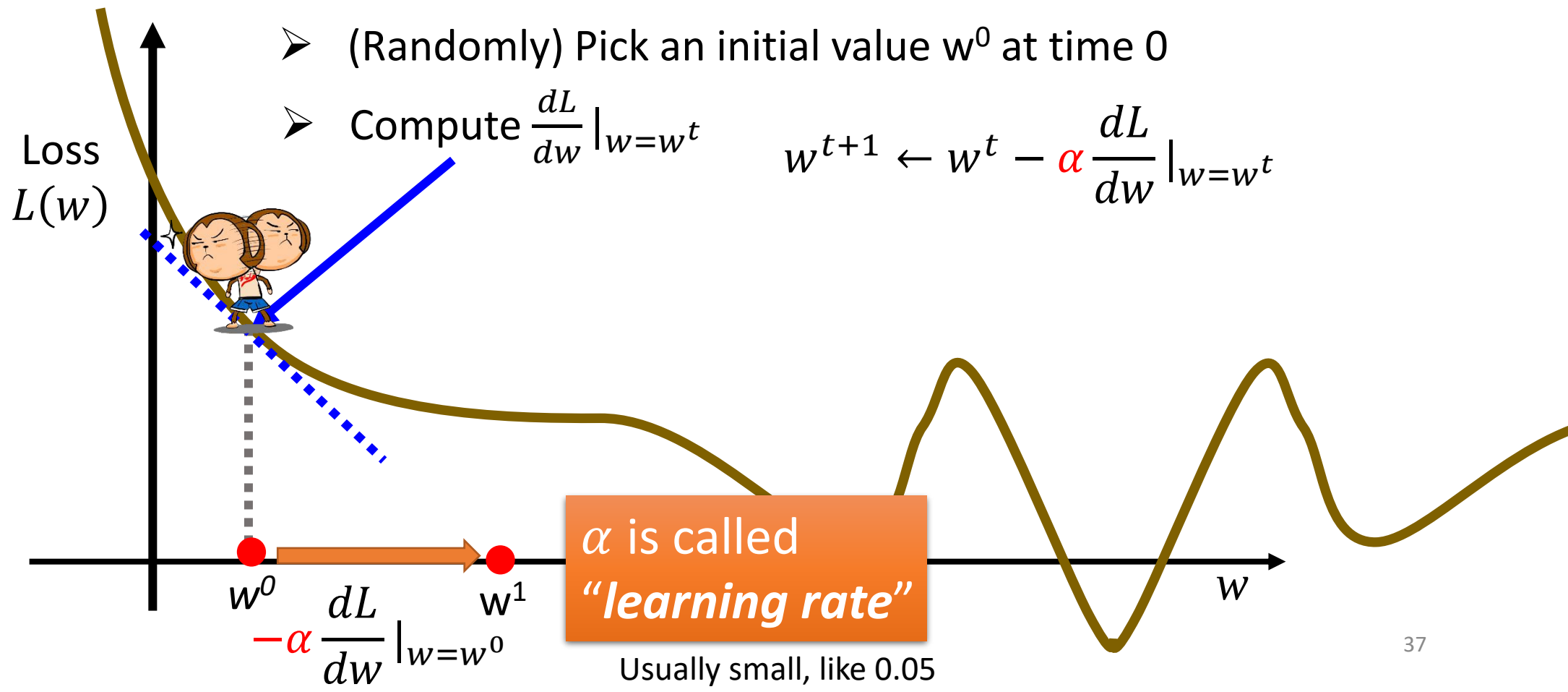
Step 3: Gradient descent

- Consider loss function $L(w)$ with one parameter w : $w^* = \arg \min_w L(w)$



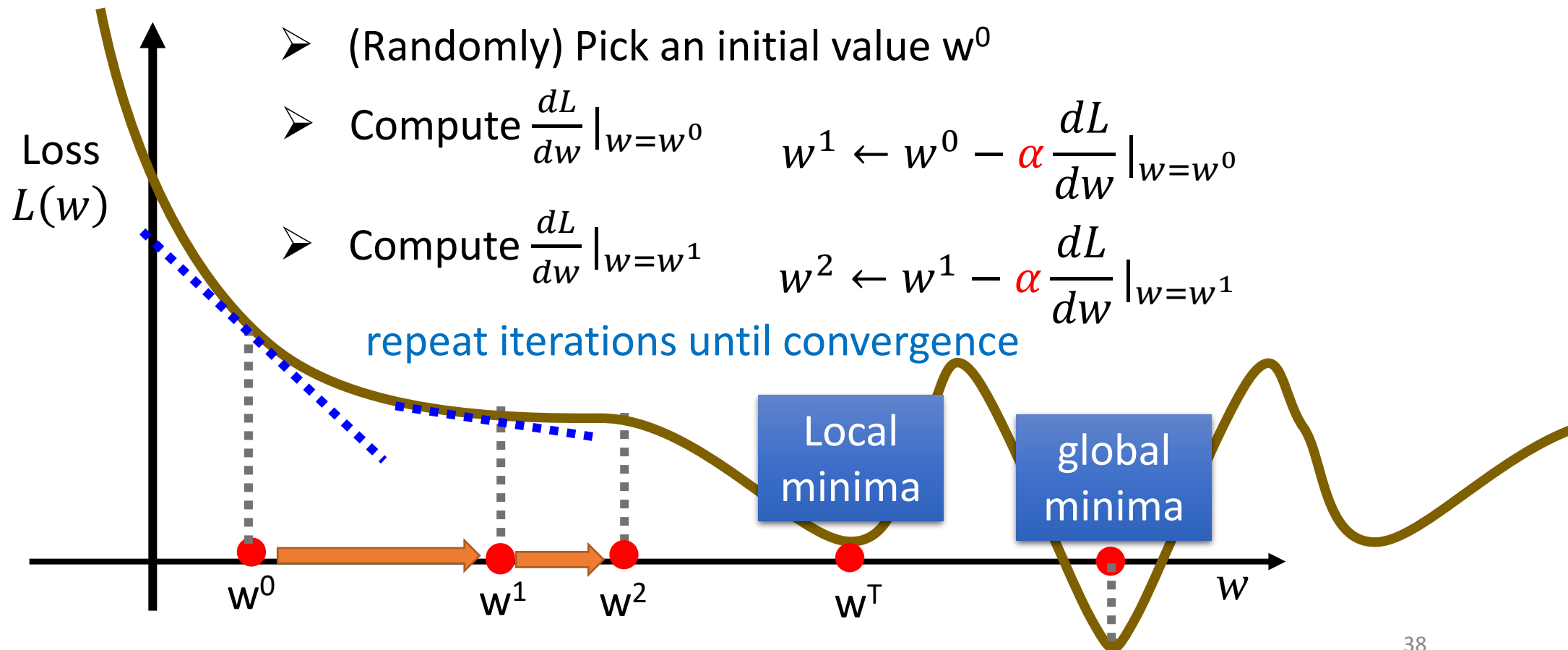
Step 3: Gradient descent

- Consider loss function $L(w)$ with one parameter w : $w^* = \arg \min_w L(w)$



Step 3: Gradient descent

- Consider loss function $L(w)$ with one parameter w : $w^* = \arg \min_w L(w)$



Step 3: Gradient descent

$$\begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix} \text{gradient}$$

- How about two parameters?

$$w^*, b^* = \arg \min_{w, b} L(w, b)$$

➤ (Randomly) Pick an initial value for each parameter w^0, b^0

➤ Compute $\frac{\partial L}{\partial w} \big|_{w=w^0, b=b^0}, \frac{\partial L}{\partial b} \big|_{w=w^0, b=b^0}$

$$w^1 \leftarrow w^0 - \alpha \frac{\partial L}{\partial w} \big|_{w=w^0, b=b^0} \quad b^1 \leftarrow b^0 - \alpha \frac{\partial L}{\partial b} \big|_{w=w^0, b=b^0}$$

➤ Compute $\frac{\partial L}{\partial w} \big|_{w=w^1, b=b^1}, \frac{\partial L}{\partial b} \big|_{w=w^1, b=b^1}$

$$w^2 \leftarrow w^1 - \alpha \frac{\partial L}{\partial w} \big|_{w=w^1, b=b^1} \quad b^2 \leftarrow b^1 - \alpha \frac{\partial L}{\partial b} \big|_{w=w^1, b=b^1}$$

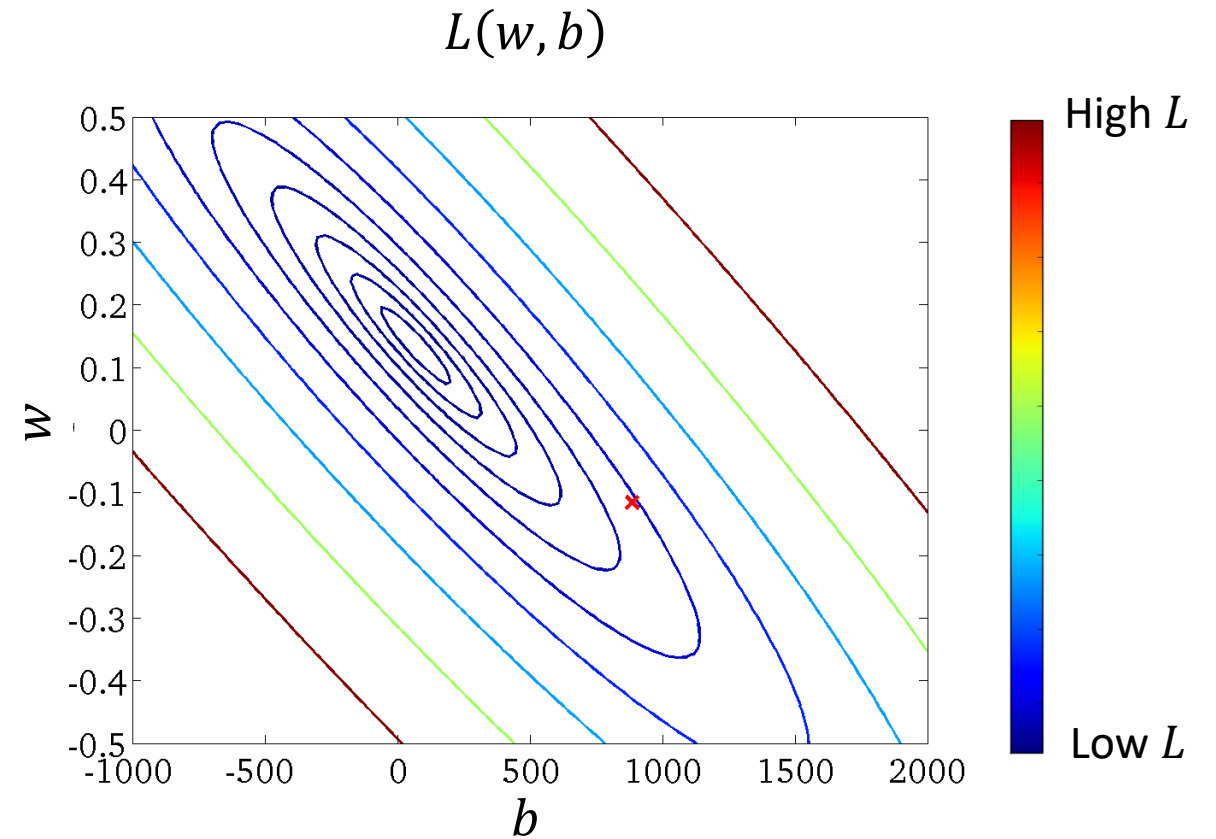
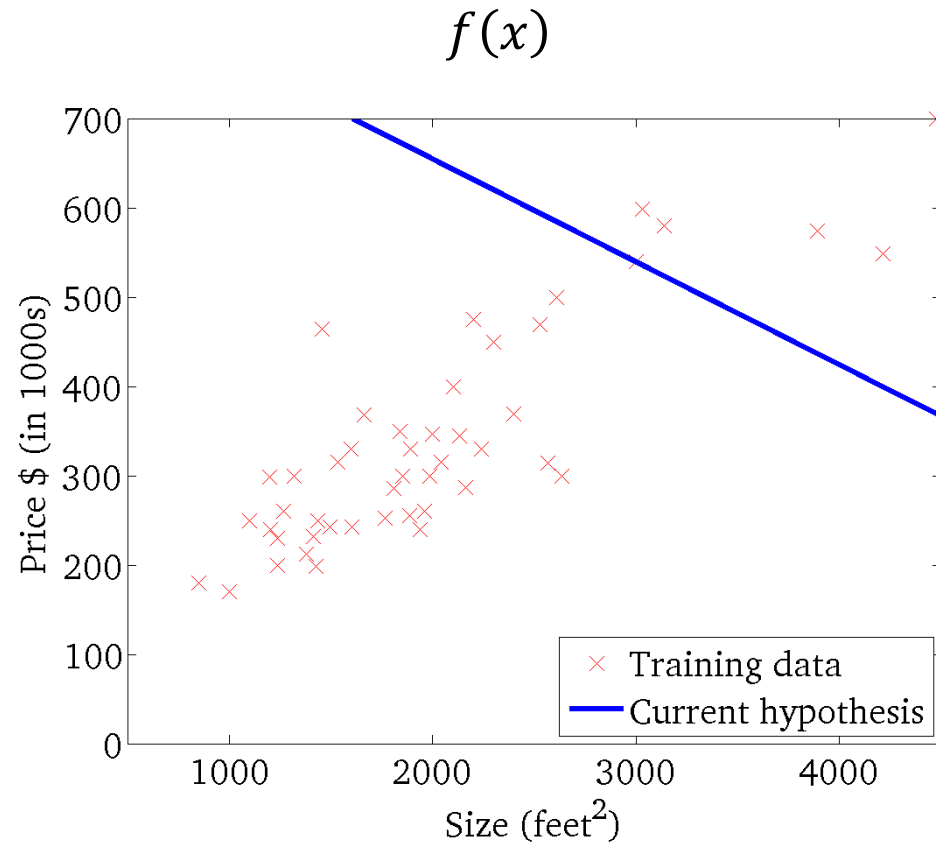
repeat iterations until convergence

Step 3: Gradient descent

- Gradient of Linear Regression

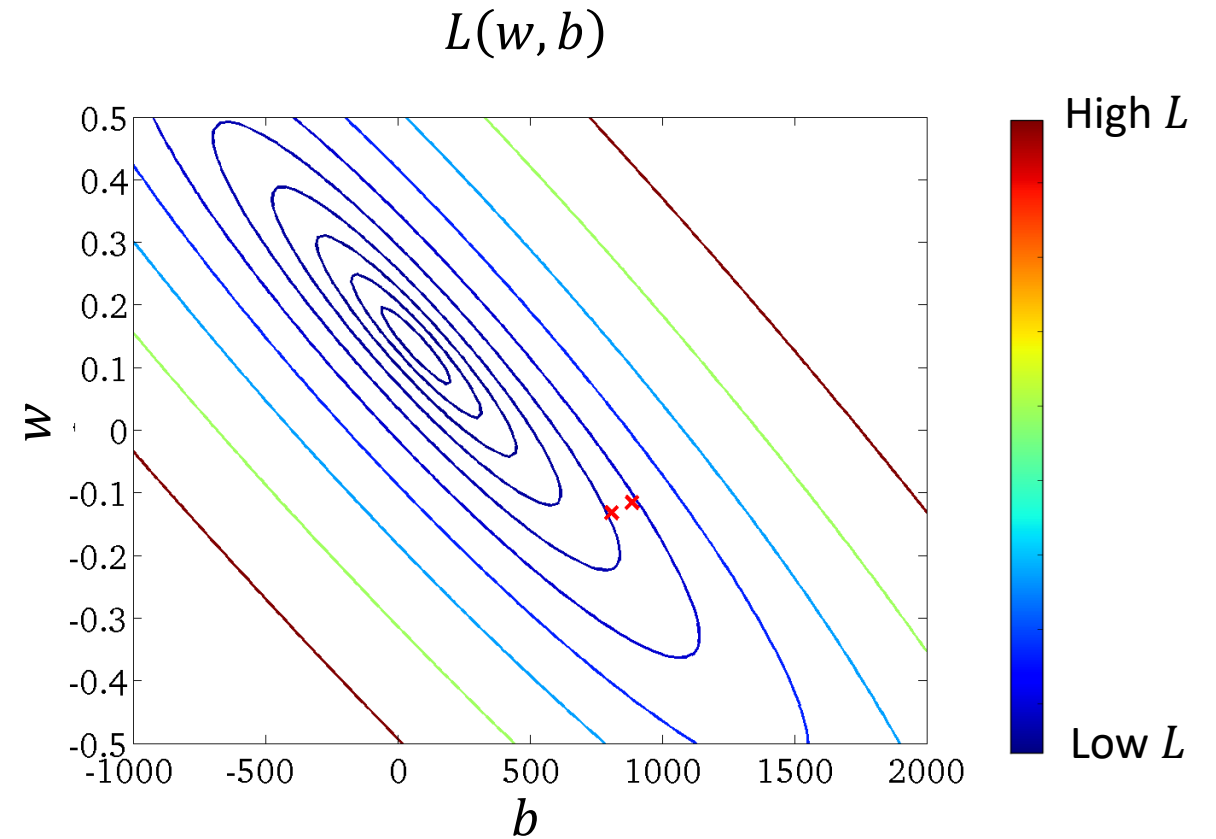
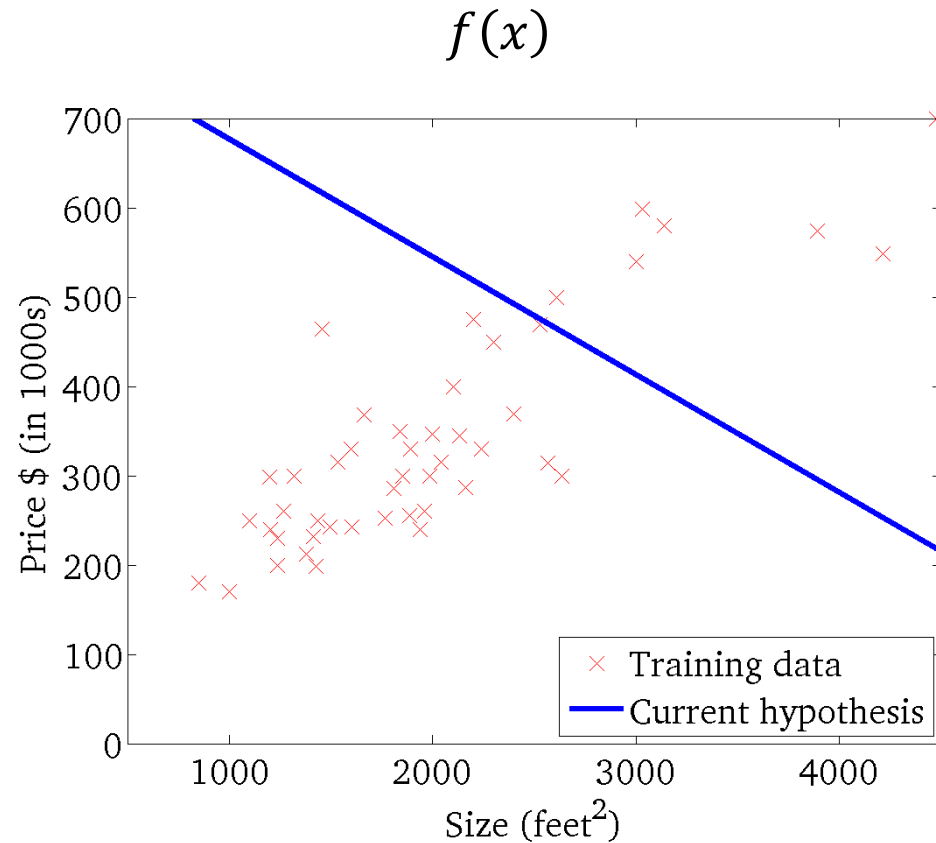
$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{\partial}{\partial w} \sum_{i=1}^n (y^{(i)} - f(x^{(i)}))^2 \\ &= \frac{\partial}{\partial w} \sum_{i=1}^n (y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)})^2 \\ &= \sum_{i=1}^n 2(y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) \times \frac{\partial}{\partial w} (y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) \\ &= - \sum_{i=1}^n 2(y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) x_j^{(i)}\end{aligned}$$

Step 3: Gradient descent



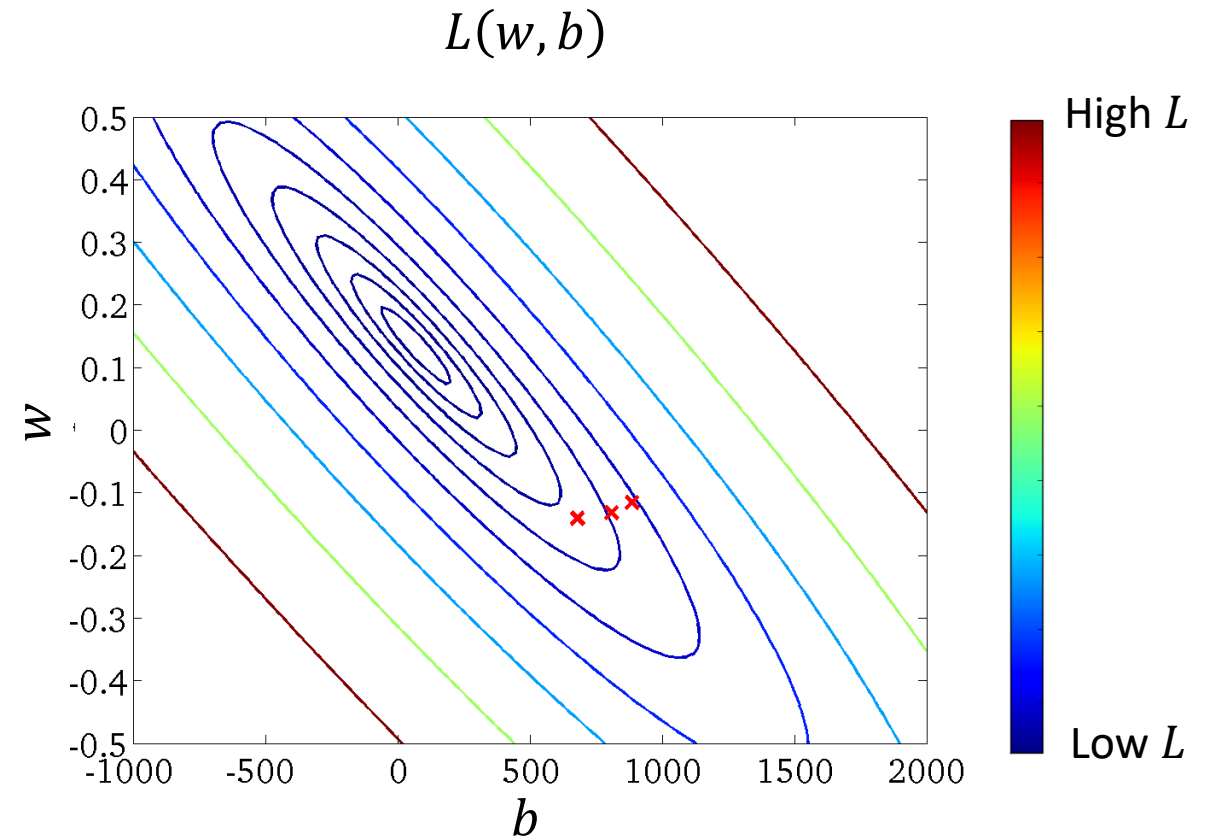
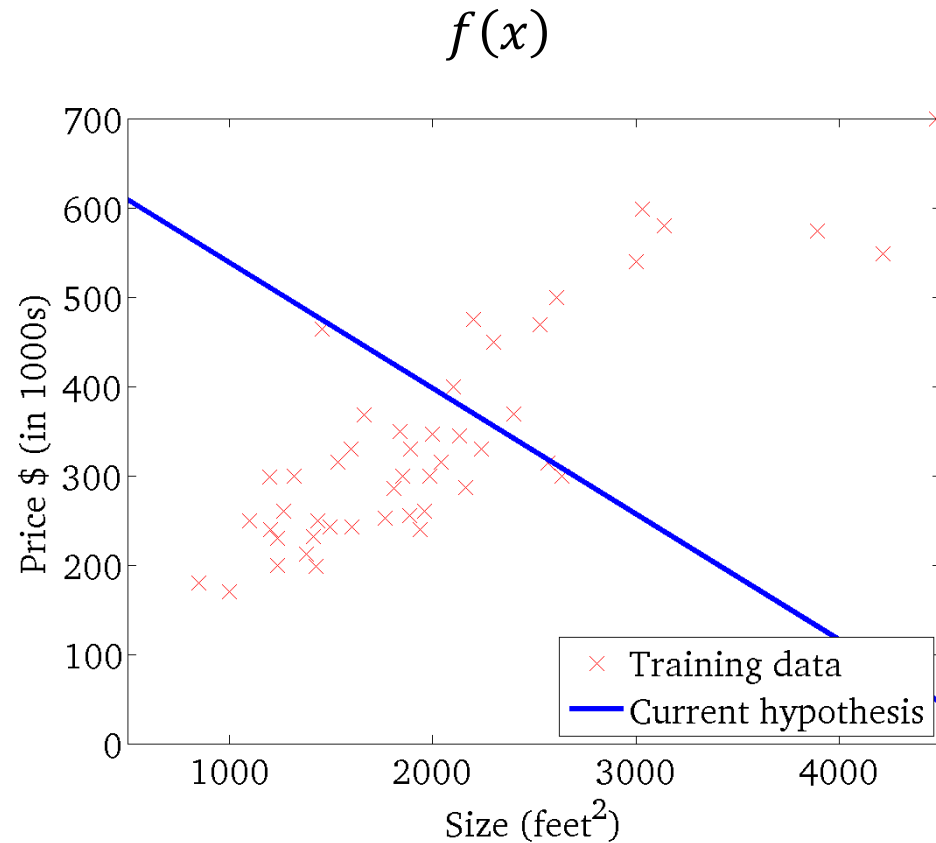
How does the loss get minimized by gradient descent

Step 3: Gradient descent



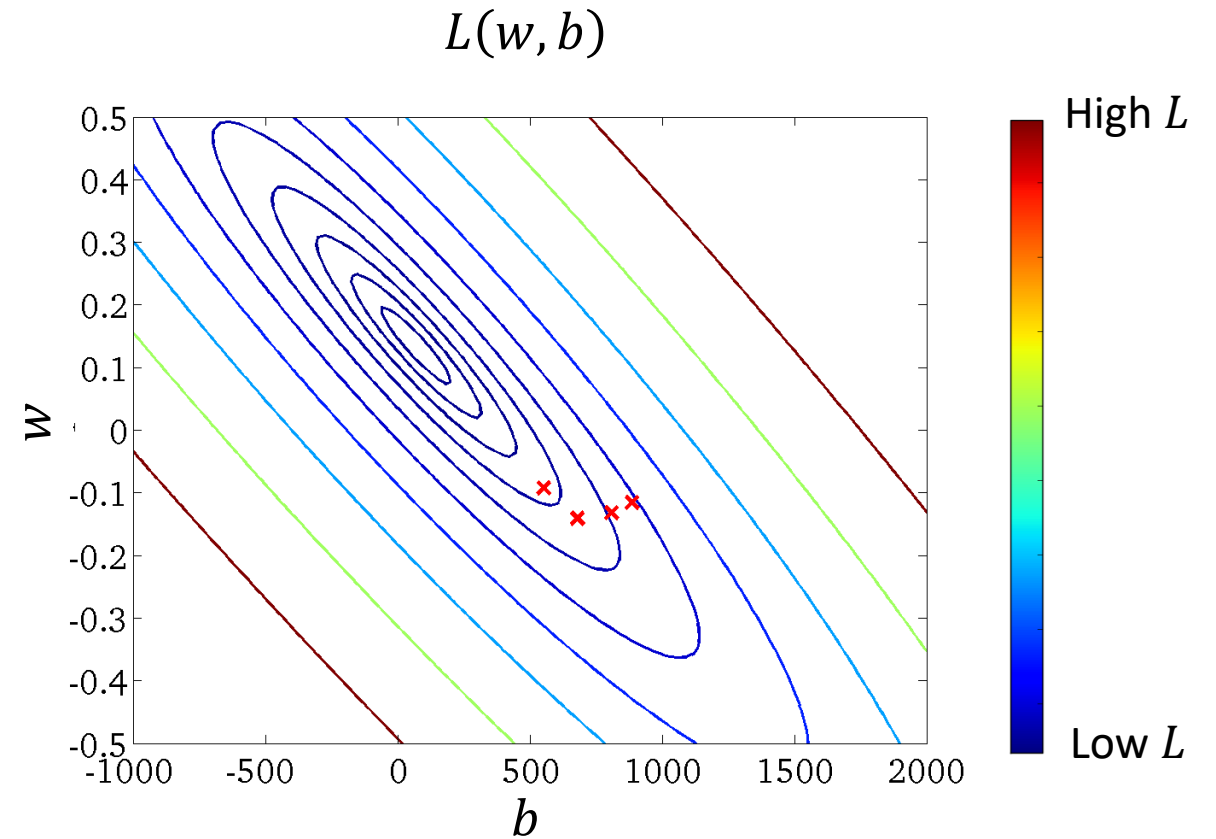
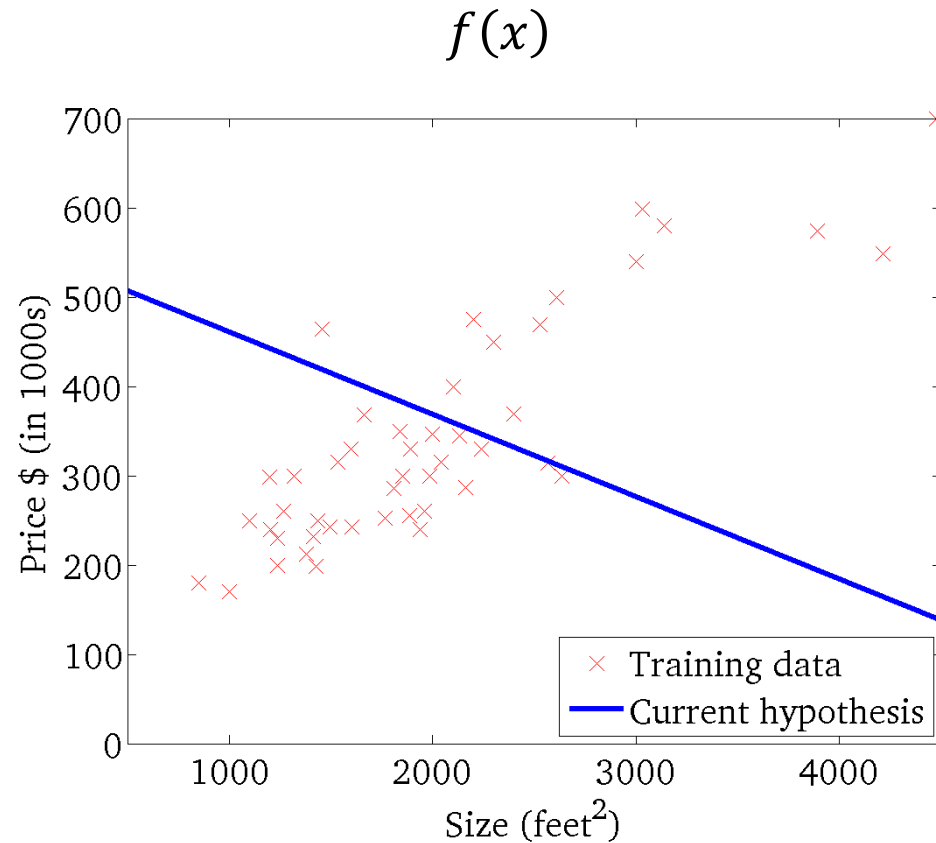
How does the loss get minimized by gradient descent

Step 3: Gradient descent



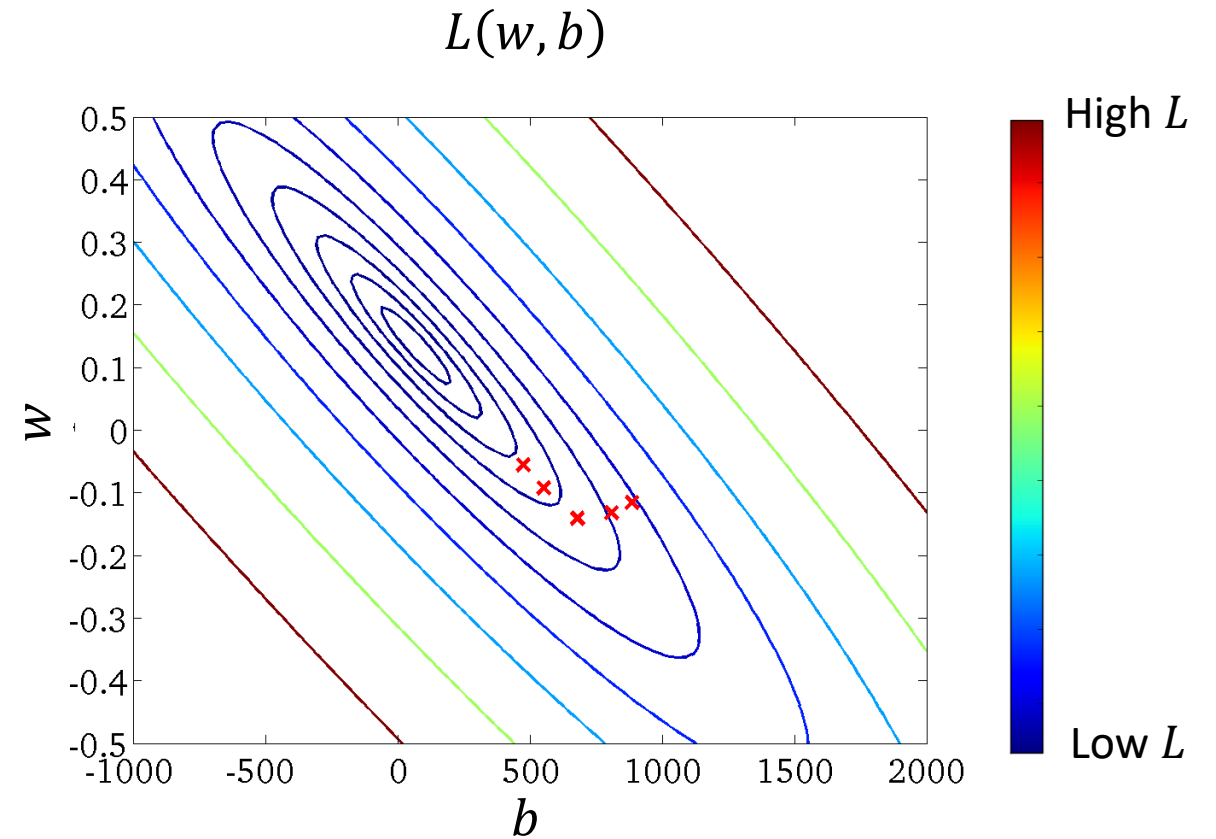
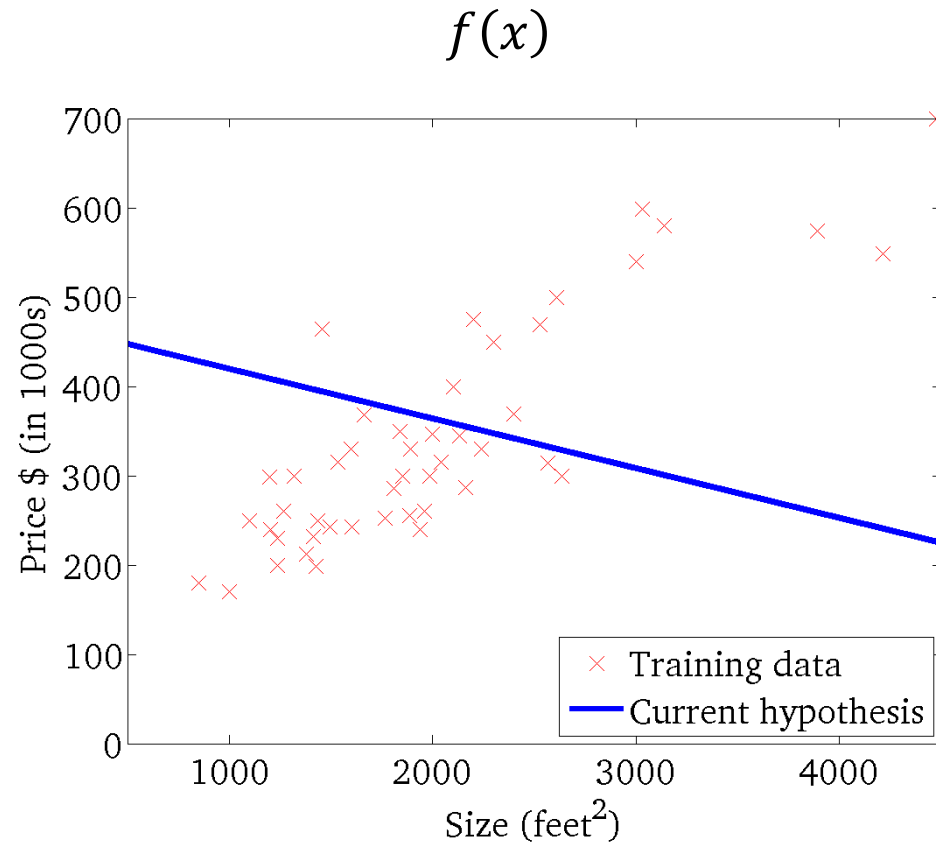
How does the loss get minimized by gradient descent

Step 3: Gradient descent



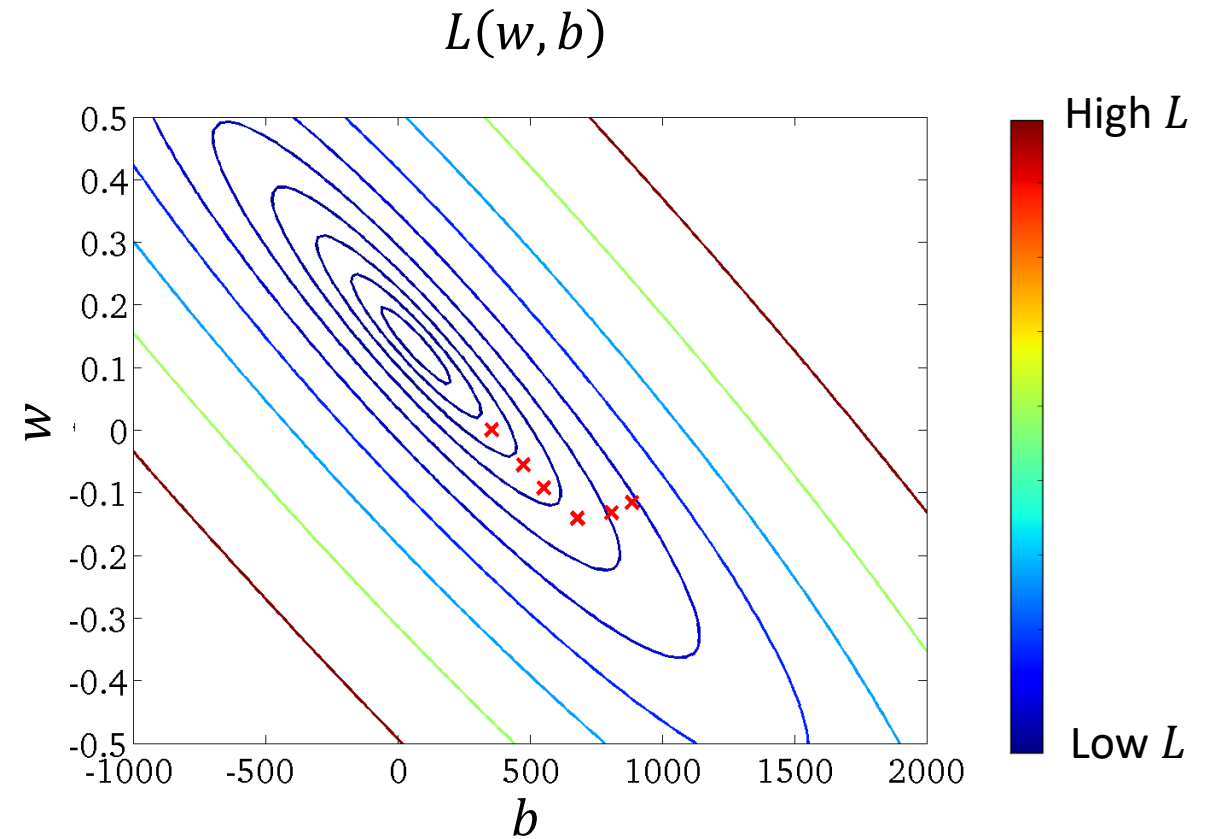
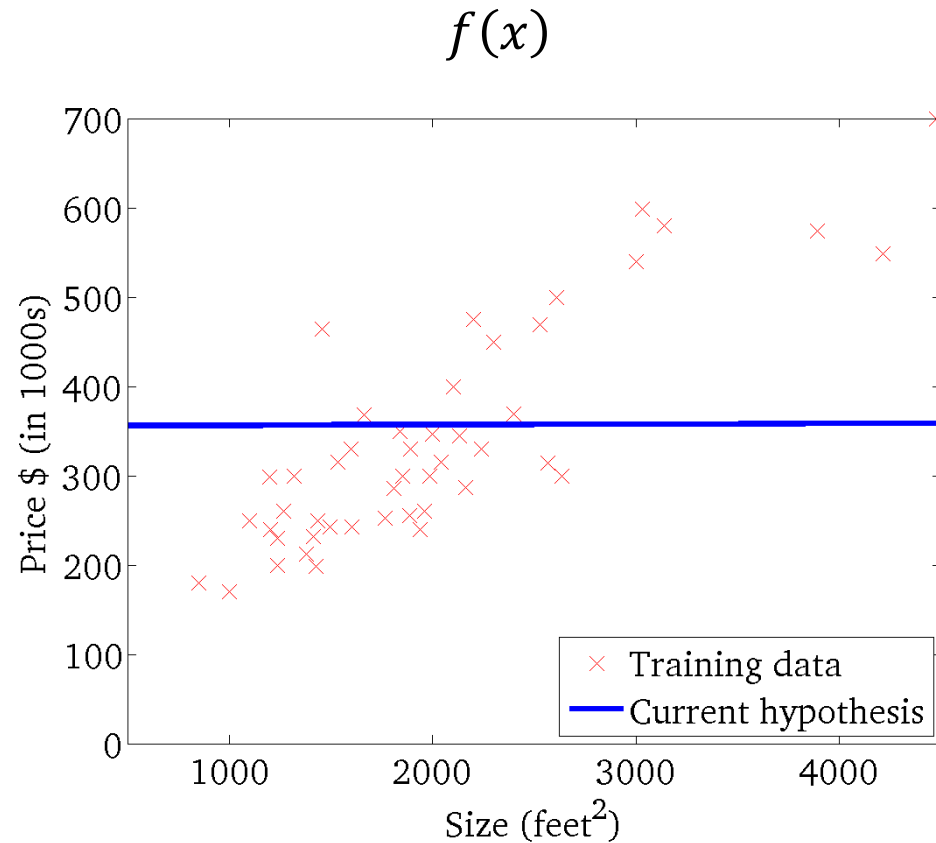
How does the loss get minimized by gradient descent

Step 3: Gradient descent



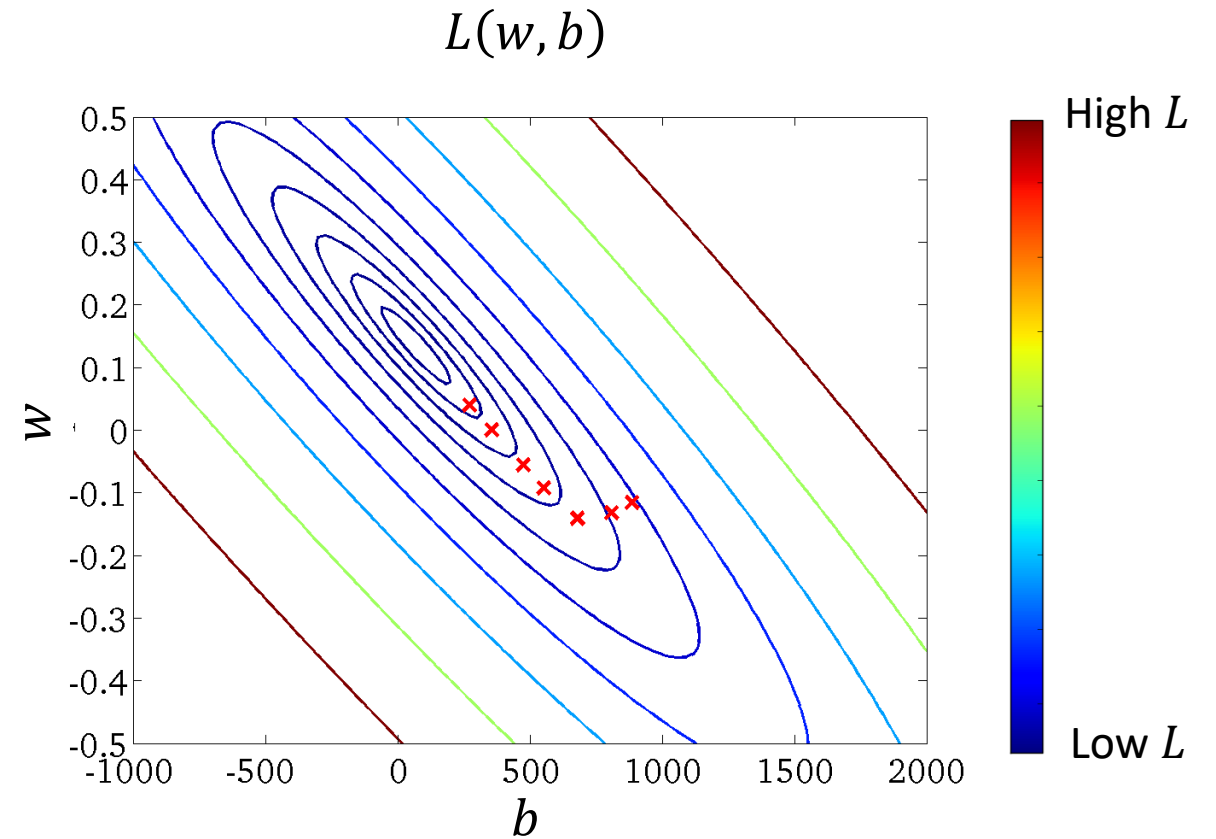
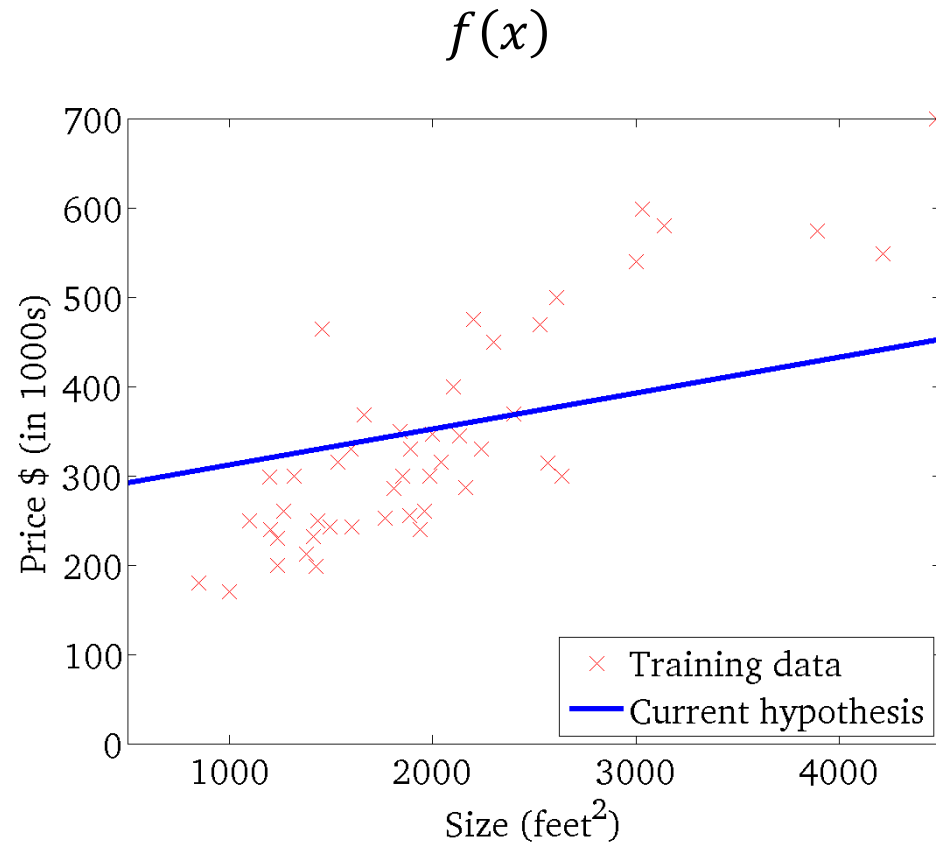
How does the loss get minimized by gradient descent

Step 3: Gradient descent



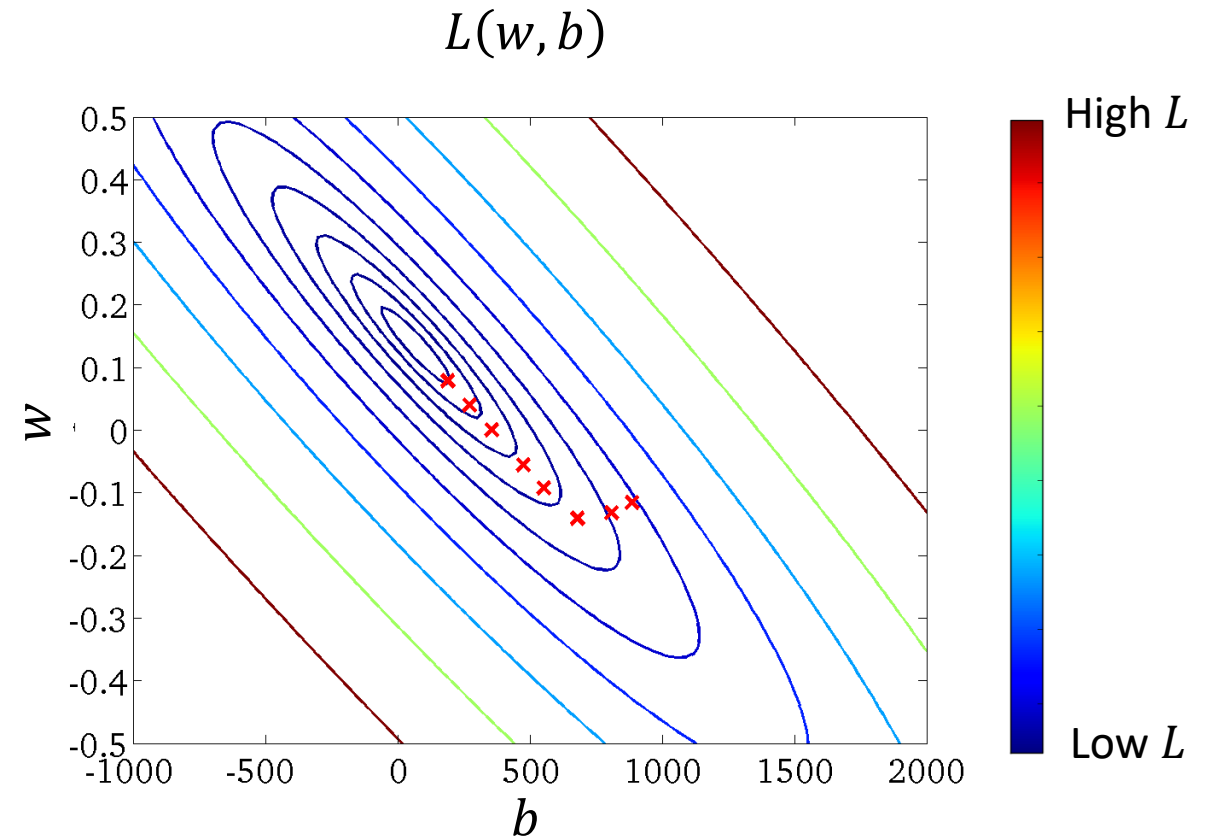
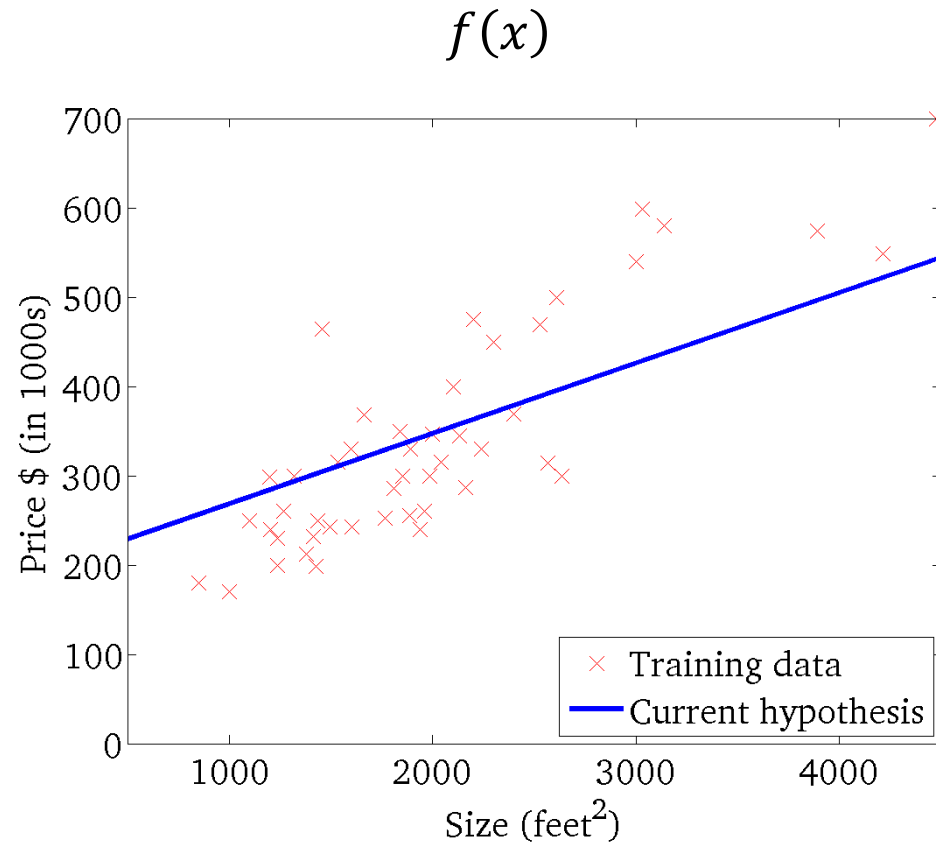
How does the loss get minimized by gradient descent

Step 3: Gradient descent



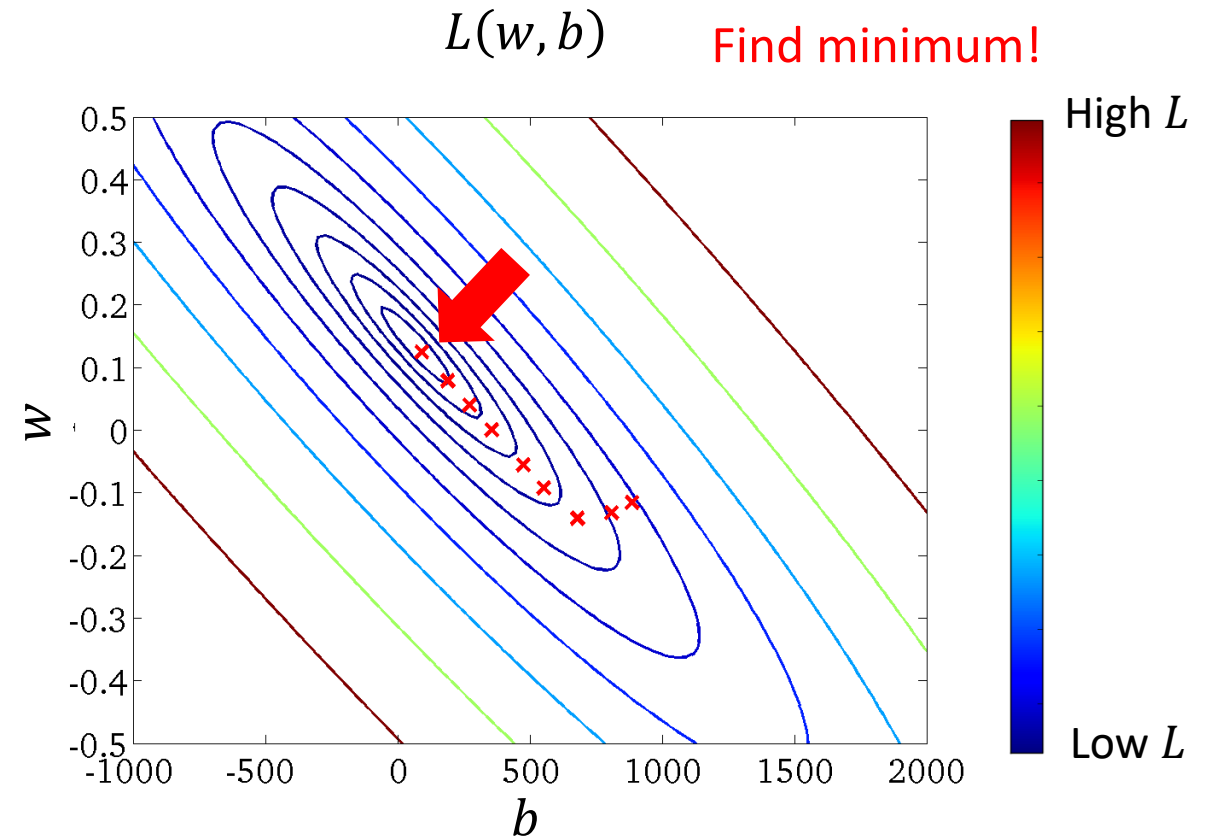
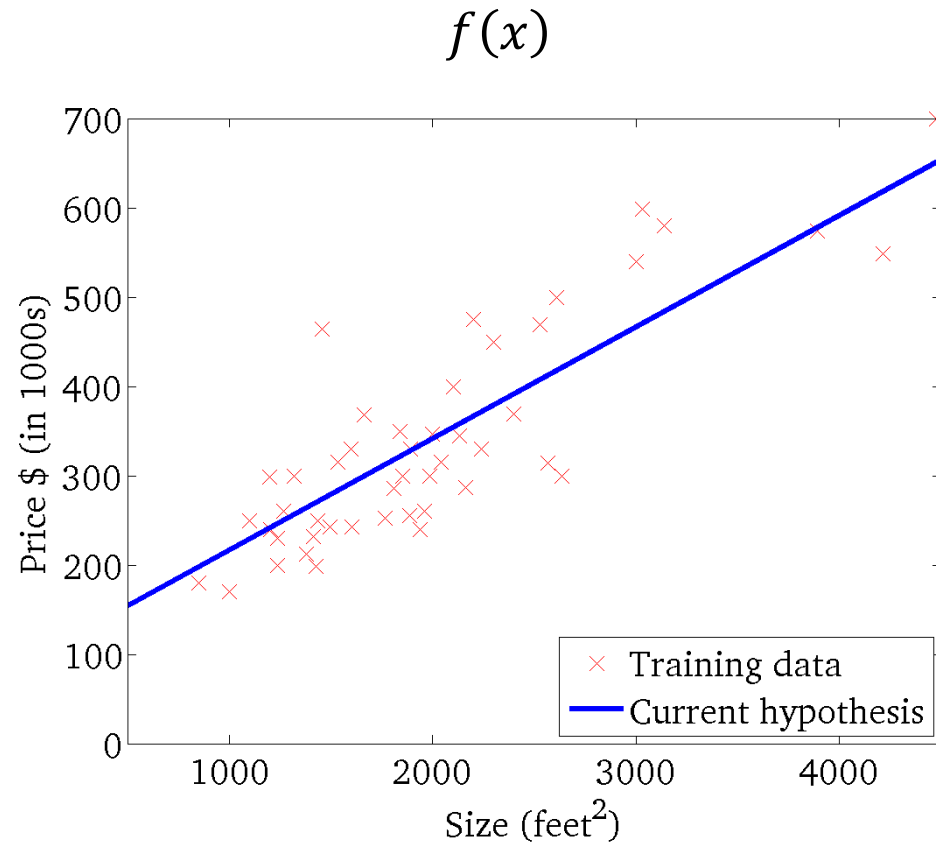
How does the loss get minimized by gradient descent

Step 3: Gradient descent



How does the loss get minimized by gradient descent

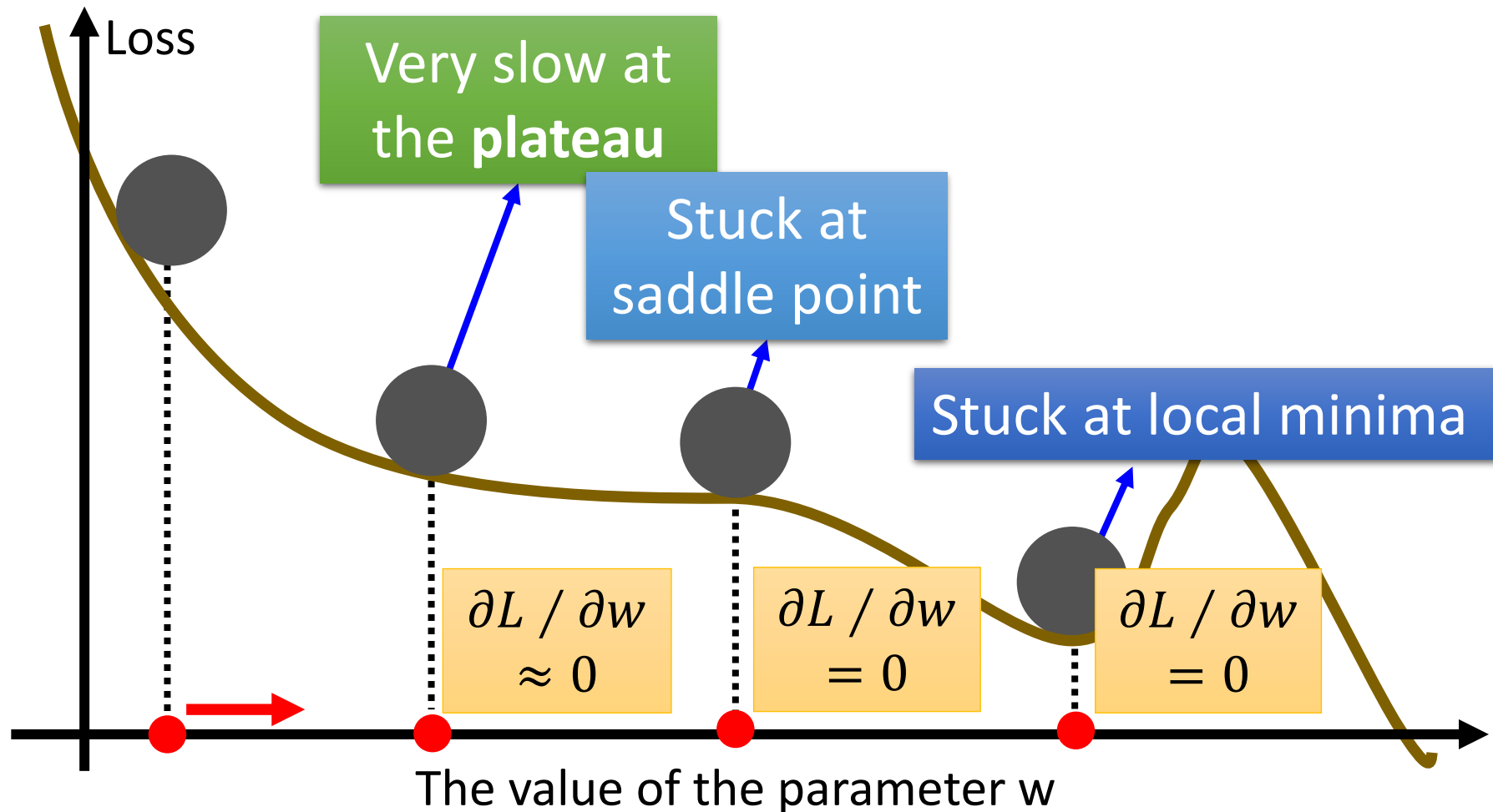
Step 3: Gradient descent



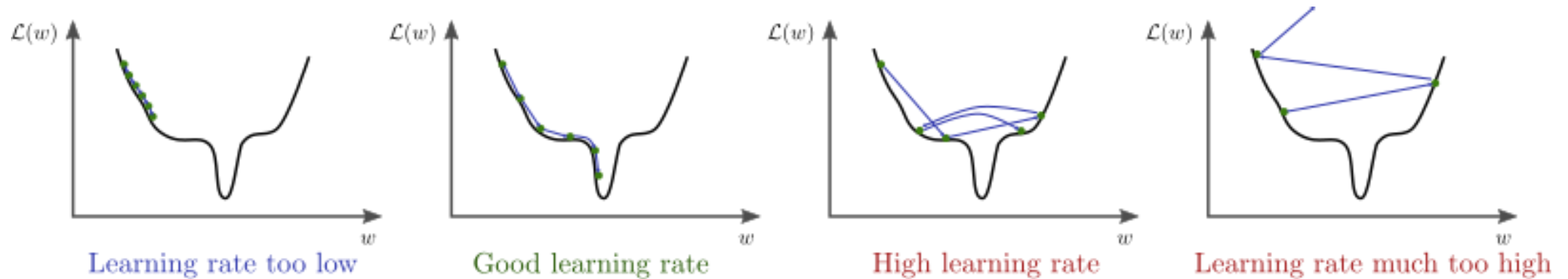
How does the loss get minimized by gradient descent

Step 3: Gradient descent

- Small gradient can slow down or halt the optimization

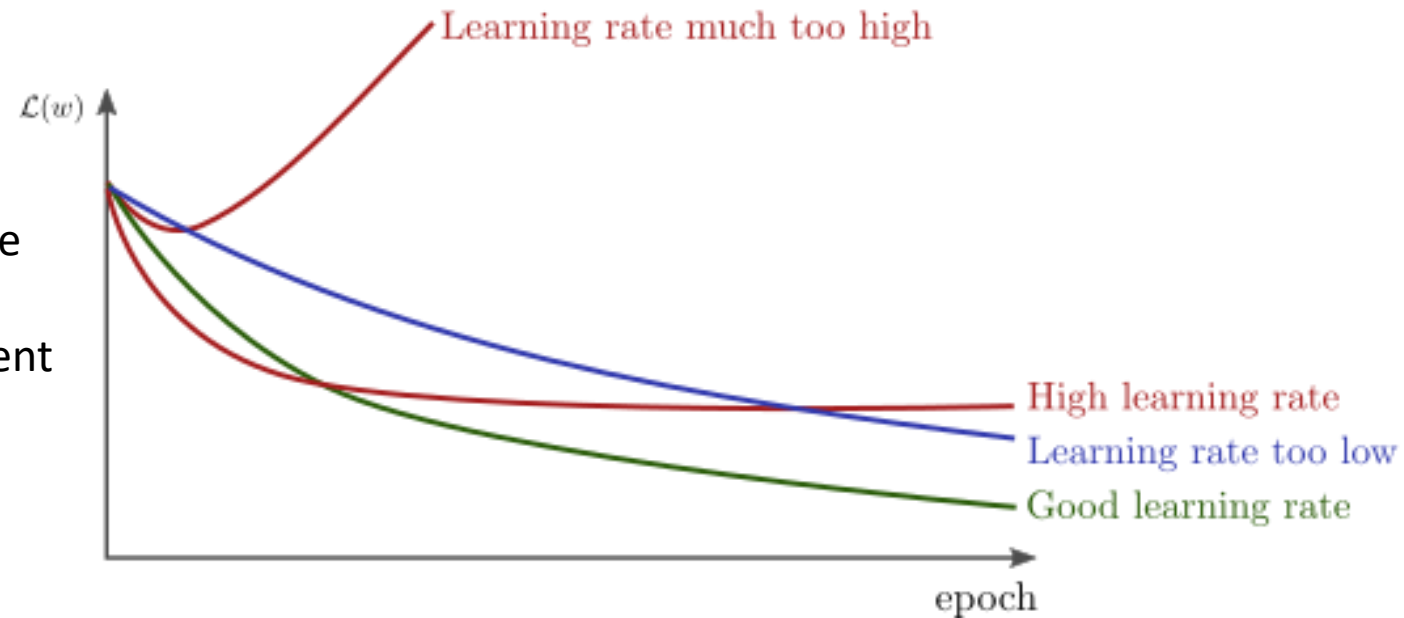


Step 3: Gradient descent – learning rate α



Monitor the loss at each iteration

- Search α by order of magnitude at first (like, 10^{-1} ... 10^{-5})
- Tune α locally to achieve efficient convergence



Linear algebra review

- Vector in $\mathbb{R}^d$ is an ordered set of d real values
- Matrix in $\mathbb{R}^{n \times m}$ is a n by m object with n rows and m columns

- Transpose $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}^T = [1 \quad 2 \quad 3 \quad 4]$ $\begin{bmatrix} 1 & 3 \\ 4 & 17 \\ 5 & 7 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 & 5 \\ 3 & 17 & 7 \end{bmatrix}$

- Matrix production

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \end{bmatrix}_{4 \times 2} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}_{2 \times 3} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \\ c_{41} & c_{42} & c_{43} \end{bmatrix}_{4 \times 3}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

Vectorization of linear regression

- Benefits of vectorization
 - More compact equations
 - Faster code (using optimized matrix libraries)

- Linear regression model:

$$f(x) = \sum_{i=0}^d w_i x_i$$

- Let $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$ $\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

- In vectorized form, linear regression model: $f(x) = \mathbf{w}^T \mathbf{x}$

Vectorization of linear regression

- Consider the model for n instance

$$f(x^{(i)}) = \sum_{j=0}^d w_j x_j^{(i)}$$

- Let

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(i)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$\mathbb{R}^{(d+1) \times 1} \qquad \qquad \mathbb{R}^{n \times (d+1)} \qquad \qquad \mathbb{R}^{n \times 1}$

- In vectorized form, linear regression model: $f(x) = \mathbf{X}\mathbf{w}$

Vectorization of linear regression

- For the loss function

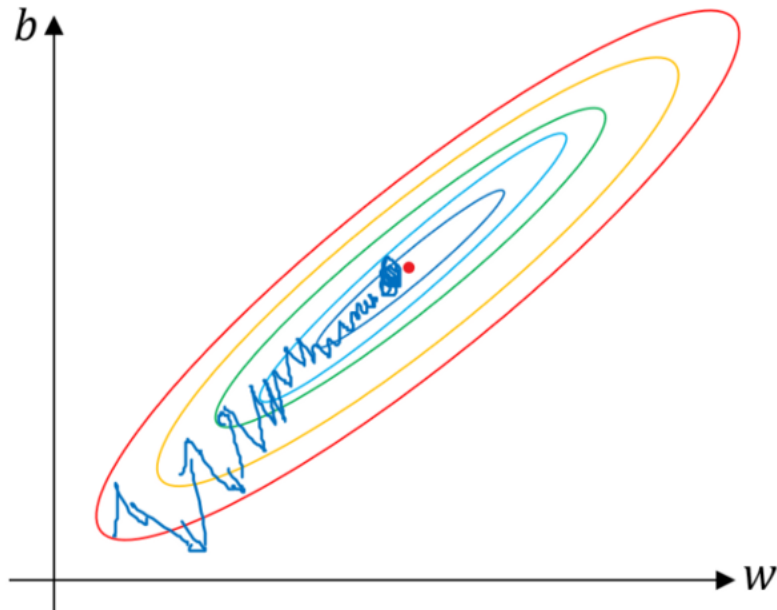
$$\begin{aligned} L(w) &= \sum_{i=1}^n (y^{(i)} - f(x^{(i)}))^2 \\ &= \sum_{i=1}^n (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2 \\ &= (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) \end{aligned}$$

One time calculation, without iterating through all data samples.

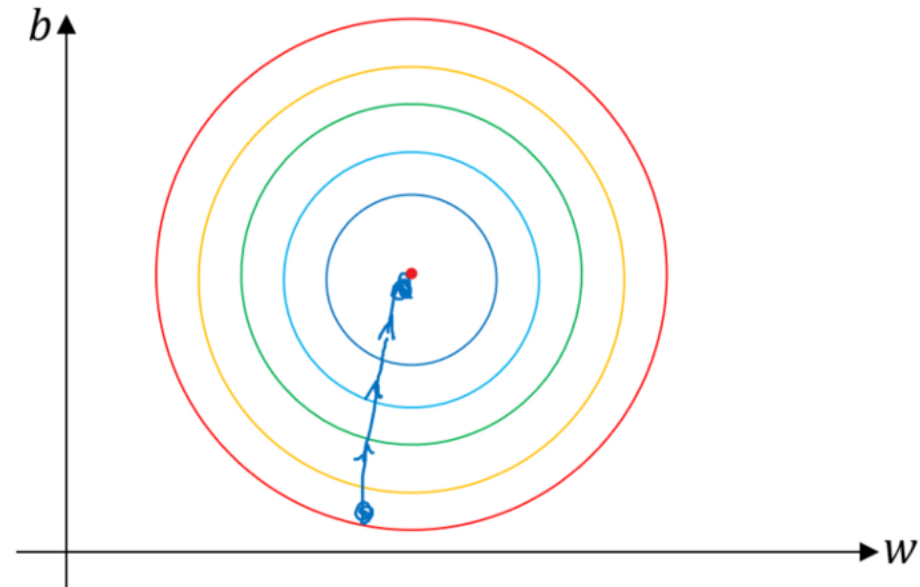
Improving learning

- Feature scaling (or normalization)
 - Ensure all features have similar scales
 - Gradient descent would converge faster

Unnormalized



Normalized



Feature standardization

- Rescale features to have zero mean and unit variance

- Let μ_j be the **mean** of feature j
$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_j^{(i)}$$

- Let s_j be the **standard deviation** of feature j
$$\sigma_j = \sqrt{\frac{\sum_{i=1}^n (x_j^{(i)} - \mu_j)^2}{n}}$$

- Replace each value with

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{\sigma_j} \quad \text{for } j = 1 \dots d \text{ (not } x_0)$$

- Must apply the same transformation for both training and testing instances
- Outliers can cause problems

Regularization

- A method to control the complexity of model, avoid overfitting
- Why - address overfitting issues by keeping w small
- How - Penalize for large value of w_j
 - Can incorporate into the loss function
 - Works well when we have a lot of features

$$L(f) = \underbrace{\sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2}_{\text{model fit to data}} + \lambda \underbrace{\sum_{j=1}^d w_j^2}_{\text{regularization}} \quad \text{Also called } L^2\text{-norm}$$

- λ is the predefined hyperparameter to control the degree of regularization
- No regularization on w_0 (bias b)

Summary

- Problem: estimate a real value
- Model: $\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$
- Loss function: sum of square (SSE)

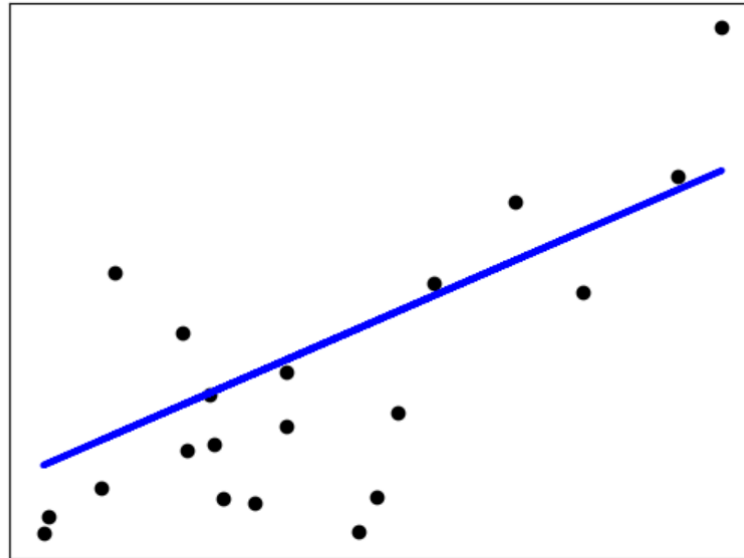
$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^i(\mathbf{w}, b) = \sum_{i=1}^n (\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)})^2$$

- Optimize parameters by Gradient Descent method
 - Choose a starting point
 - Repeat
 - Compute gradient
 - Update parameters

Demo

- Use ML library
 - https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols.html

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True, normalize='deprecated', copy_X=True, n_jobs=None, positive=False)
```



Out: Coefficients:
[938.23786125]
Mean squared error: 2548.07
Coefficient of determination: 0.47